CS336 作业 5：对齐（Alignment）

原文：Stanford CS336 作业说明（英文原版）

本文档为英文原版作业说明的非官方中文翻译，仅供学习参考；如与原文有歧义，请以英文原文为准。

目录

1 作业概览 **3**

2 引言 **4**

2.1 背景 4

2.2 模型与数据集 5

2.3 记号 5

3 提示 **6**

3.1 使用 vLLM 进行推理 7

3.2 零样本、少样本与思维链提示 7

3.3 评分函数 8

3.4 实验 9

4 组相对策略优化 **10**

4.1 推导在策略 GRPO 10

4.1.1 将语言模型视为策略 10

4.1.2 轨迹 11

4.1.3 奖励与回报 11

4.1.4 策略梯度 11

4.1.5 基线 12

4.1.6 优势归一化 14

4.1.7 序列归一化 14

4.1.8 整合所有组件 15

4.2 实现在策略 GRPO 16

4.2.1 使用 Hugging Face 模型 16
4.2.2 在强化学习循环中使用 vLLM 18
4.2.3 GRPO 组件 19
4.2.4 GRPO 训练步 23
4.3 实验 27

5 RL 算法变体 **29**
5.1 Dr. GRPO 29
5.2 拒绝微调 30
5.3 MaxRL 31
5.4 实验 33

6 离策略 RL **34**
6.1 重要性重加权 35
6.2 PPO/GRPO 风格的重要性重加权与裁剪 35
6.2.1 token 级重加权 35
6.2.2 裁剪 37
6.3 GSPO 38
6.4 实验 39

7 尝试你自己的策略梯度估计量 **41**

1 作业概览

在本作业中，你将获得一些动手经验：训练一个语言模型进行推理（reasoning）并解决下游任务。

你将实现的内容

1. 零样本（zero-shot）、少样本（few-shot）与思维链（chain-of-thought）提示。
2. 组相对策略优化（Group Relative Policy Optimization，GRPO），一种强化学习算法，用于在给定外部奖励的情况下提升模型性能。
3. 策略梯度估计的若干变体，以探索方差缩减与重要性权重裁剪（importance weight clipping）策略。

你将运行的内容

1. 在 GSM8K 上测量 OLMo-2-0425-1B 的提示（prompting）性能。
2. 在 OLMo-2-0425-1B 上运行在策略（on-policy）GRPO，以提升其在 GSM8K 上的性能。
3. 运行多种 RL 变体，包括 RFT、Dr. GRPO 与 MaxRL，以探索 RL 中的算法选择。
4. 运行离策略（off-policy）GRPO，以加速训练并探索各种裁剪策略。

代码仓库的结构

作业的全部代码以及本说明文档都可以在 GitHub 上获取：https://github.com/stanford-cs336/assignment5-alignment。请 `git clone` 该仓库。如果有任何更新，我们会通知你，你可以通过 `git pull` 获取最新版本。

1. `cs336_alignment/*`：这里是你编写作业 5 代码的地方。注意这里没有任何现成代码（除了下文所述的起始代码之外），所以你应当可以从零开始自由实现。
2. `cs336_alignment/vllm_utils.py`：该文件包含启动一个能生成文本并同步权重的 vLLM 服务器的代码。
3. `cs336_alignment/drgrpo_grader.py`：该文件包含为数学题的模型输出评分的代码。
4. `cs336_alignment/prompts/*`：为方便起见，我们提供了若干包含提示（prompt）的文本文件。
5. `tests/*.py`：该目录包含必做的 GRPO 测试以及可选的对齐/安全补充（alignment/safety supplement）测试。

`tests/test_grpo.py` 中的必做测试会调用 `tests/adapters.py` 中定义的钩子（hook）。你需要实现这些适配器（adapter），把你的代码接入测试。多写一些测试和/或修改测试代码对调试你的代码很有帮助，但你的实现需要通过最初提供的测试套件。

1. `README.md` ：该文件包含一些搭建环境的基本说明。

如何提交

你需要向 Gradescope 提交以下文件：

- `writeup.pdf` ：回答所有书面问题。请对你撰写的回答进行排版。
- `code.zip` ：包含你写的全部代码。

运行 `test_and_make_submission.sh` 中的脚本即可生成 `code.zip` 文件。

2 引言

2.1 背景

在本课程的前四次作业中，我们学习了如何预训练一个基础模型（base model）。现在我们准备学习后训练（post-training）：一旦我们有了基础模型，如何把它变成一个能解决下游任务的有用工具？

后训练的一个组成部分是对齐（alignment）。预训练由于其数据混合（data mixture）与训练目标，会让模型具备广泛的知识与行为。但当我们要求模型解决任务时，我们想要的是一种特定行为，即做一个有帮助且无害的助手。把一个宽泛的基础模型改造成一个专注的对话模型（chat model）的过程称为“指令微调（instruction tuning）”或“对齐（alignment）”，我们将在可选的作业 5 补充材料（Assignment 5 supplement）中介绍这些技术。

后训练的另一个组成部分是强化学习（reinforcement learning）。在预训练中，由于我们的目标是让模型具备广泛的知识基础，我们使用广泛的数据（网络上的文本）。而在后训练 RL 中，我们的目标变窄了：我们希望模型在某个特定任务上达到很高的准确率，比如解数学题。这一更窄的目标与预训练至少有两点不同：(a) 我们不再拥有那么多数据；(b) 我们的训练目标从“覆盖”（拥有广泛的知识基础）变成了“精准”（产生准确的回答）。由于这些差异，我们需要一种新的技术，即强化学习（RL）。

在 RL 中，我们会得到一个由问题组成的数据集，以及一个判断某个回答是否正确解决问题的评分函数。例如，在编程场景中，我们可能拿到一道编程题（例如“写一个反转列表的 Python 函数”），以及一个由一组测试用例构成的评分函数（例如 `assert f([0, 1, 2]) == [2, 1, 0]` ）。又如在数学场景中，我们可能拿到一道数学推理应用题（例如“如果四年前 Tom 的年龄是 John 的一半，而 John 现在二十岁，Tom 现在多大？”），以及一个解析最终答案并检查其是否正确（12）的评分函数。值得注意的是，与预训练不同，我们并没有拿到可供模仿的回答数据集：在编程的例子中，我们没有拿到解决该任务的正确 Python 程序；在数学推理的例子中，我们没有拿到能得出答案的正确推理链。相反，在 RL 中，我们直接以模型准确率作为目标函数来做梯度步。从高层次来看，RL 包括：从模型采样回答、用评分函数给它们打分，然后上调那些正确回答的权重。

在本作业中，我们将从数学和实验两个角度学习 RL。事实证明 RL 很难：它又慢又不稳定。为 RL 做科学也很难：RL 实验在不同随机种子之间方差很大，而且实现中看似微小的细节影响巨

大。本作业将介绍 LLM RL，并探索其中的一些挑战。

2.2 模型与数据集

本作业将使用 OLMo-2-0425-1B，一个在 OLMo-mix-1124 上预训练、并在 Dolmino-mix-1124 上做中期训练（mid-trained）的基础模型，总共使用了 4 万亿（4 trillion）个 token。OLMo-mix-1124 主要由 DCLM-Baseline [J. Li et al., 2024] 组成（我们在课上讲过这个数据集），而 Dolmino-mix-1124 是一个更聚焦的数据混合，大约由 50% 的 DCLM 数据和 50% 的指令跟随、数学、代码、STEM 论文以及 wiki 数据组成。OLMo-2-0425-1B 的其他细节可参阅 OLMo 2 技术报告 [T. OLMo et al., 2024]。学到课程的这个阶段，你应当已经具备理解他们每一个设计决策所需的全部背景知识。

作为下游任务，我们将使用 GSM8K 数据集 [K. Cobbe et al., 2021]，它可以在作业仓库的 `data/gsm8k/train.jsonl` 和 `data/gsm8k/test.jsonl` 找到，也可以在此处在线获取。该数据集包含一组难度较低的小学数学推理应用题。下面是一个例子：

```
{
  "question": "Natalia sold clips to 48 of her friends in April, and then she sold half as many
    clips in May. How many clips did Natalia sell altogether in April and May?",
  "answer": "Natalia sold 48/2 = <<48/2=24>>24 clips in May.\nNatalia sold 48+24 = <<48+24=72>>72
    clips altogether in April and May.\n#### 72"
}
```

在 RL 过程中，我们的模型将学会为这类题目生成推理链，从而提升其解决数学问题的能力。

关于模型与数据集选择的说明：受资源所限，我们只选择了一个小规模的“模型-数据集”组合来充当我们的 RL 测试床（testbed）。具体而言，我们选择的这个小模型相当能干，因为它在极大量的 token 上训练过（是参数量的 4000 倍！），这让我们能够在一个真实数据集上、以小规模看到合理的 RL 结果。遗憾的是，我们在这门课上训练的模型太弱，解不了数学题。如果你在完成本作业后还有兴趣，也欢迎把你在这门课上训练的模型拿到一个更简单的任务上跑 RL。由于 RL 的训练动态非常依赖于具体的模型和数据集，请记住：我们在本作业中看到的结果未必能迁移到其他“模型-数据集”组合上。

2.3 记号

本作业会涉及一些数学，因此我们在下表中给出后面会用到的部分记号。语言建模与强化学习常常对同一个对象使用不同的术语，所以表中同时列出了两种术语。如果你忘了某个记号指的是什么，可以随时回到这张表。

记号	语言建模术语	强化学习术语	含义
ρ	提示分布（prompt distribution）或数据集	初始状态分布（initial state distribution）	提示/问题上的一个分布。

记号	语言建模术语	强化学习术语	含义
x	提示；问题（prompt; question）	初始状态（initial state）	从 ρ 中采样出的提示/问题。
y	回答；补全；生成；样本（response; completion; generation; sample）	rollout；轨迹（trajectory）；采样的动作序列	对提示 x 采样出的一个回答。
y_t	token	动作（action）	y 中第 t 个生成的 token。
$y_{<t}$	前缀（prefix）		位置 t 之前所有已生成的 token，即 $y_1, \ldots, y_{t-1}$。
π_θ	模型（model）	策略（policy）	参数为 θ 的模型，它在给定提示 x 的条件下为回答 y 分配概率 $\pi_\theta(y \mid x)$。
$\pi_\theta(y_t \mid x, y_{<t})$	下一 token 分布（next-token distribution）	时间步 t 处的策略（policy at timestep t）	在给定提示与之前已生成 token 的条件下，token y_t 的条件概率。
$r(y \mid x)$		奖励（reward）	表示采样回答正确与否的标量分数（本作业中为 0 或 1）。
B	每批的提示数（number of prompts per batch）		每个推理批次（inference batch）中的提示数量。
G	每个提示的生成数（generations per prompt）	组大小（group size）	为每个提示采样的回答数量。
$\text{len}(y)$ 或 L	回答长度（response length）	视界（horizon）	一条回答中生成的 token 数量。
$A^{(i,j)}$		优势（advantage）	经过基线（baseline）减除与归一化之后，分配给提示 i 的回答 j 的权重。

3 提示

我们将预训练基础模型应用于下游任务的第一步是对它进行提示。基础模型在预训练时见过各种各样的行为，而提示是一种轻量级的方式，可以把模型的行为引导到解题模式上。在本作业的后半部分，我们还会看到提示的选择会影响 RL 的动态与探索。

给模型提示的最基本方式是只提供问题，然后从模型的下一 token 分布中采样来产生答案；我们把这种策略称为 `question_only` 。我们将把这种策略与 `r1_zero` 提示进行比较，后者既包含问

题，也包含让模型进行思维链（chain-of-thought）推理的指令 [DeepSeek-AI et al., 2025]。

3.1 使用 vLLM 进行推理

要从模型生成回答，我们需要一个推理引擎。实现一个推理引擎超出了本作业的范围，因此我们将使用 vLLM 推理引擎 [W. Kwon et al., 2023]，它实现了多种优化，包括快速的 CUDA kernel、用于高效注意力 KV 缓存的 PagedAttention 等等。启动 vLLM 服务器并生成文本的代码已在 `cs336_alignment/vllm_utils.py` 中提供，其接口如下：

```
@dataclass
class VLLMCompletion:
    text: str
    token_ids: list[int]
    finish_reason: str | None

@dataclass
class VLLMServer:
    model_id: str
    gpu: int = 0
    seed: int = 0
    gpu_memory_utilization: float = 0.9
```

```
    def start(self) -> None: ...

    def generate_completions(
        self,
        prompts: list[str],
        sampling_params: dict,
        batch_size: int | None = None,
    ) -> list[VLLMCompletion]: ...
```

3.2 零样本、少样本与思维链提示

除非另有说明，在 GSM8K 上的实验我们将使用来自 DeepSeek R1-Zero 模型 [DeepSeek-AI et al., 2025] 的如下提示。我们将其称为 `r1_zero` 提示：

```
A conversation between User and Assistant. The User asks a question, and the Assistant solves it.
    The Assistant first thinks about the reasoning process in the mind and then provides the User
    with the answer. The reasoning process is enclosed within <think> </think> and the answer is
    enclosed within <answer> </answer> tags, respectively, i.e., <think> reasoning process here </
    think> <answer> answer here </answer>.
User: {question}
Assistant: <think>
```

`r1_zero` 提示位于文本文件 `cs336_alignment/prompts/r1_zero.prompt` 中。

在提示中，`question` 指我们插入的某个问题（例如 “Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in

April and May?”)。我们的期望是：模型扮演助手（assistant）的角色，开始生成思考过程（因为我们已经预先放入了起始的 <think> 标签），用 </think> 结束思考过程，然后在 answer 标签内生成一个最终的符号答案，例如 <answer> 4x + 10 </answer> 。让模型生成 <answer> </answer> 这类标签的目的是：这样我们就能方便地解析模型的输出并与标准答案（ground truth）比较，并且当我们看到结束标签 </answer> 时，可以停止生成回答。

另一种提示方法称为少样本提示（few-shot prompting），即在把实际问题喂给模型之前，先在前面附上若干问答对。r1_zero 提示的少样本版本形如：

```
A conversation between User and Assistant. The User asks a question, and the Assistant solves it.
    The Assistant first thinks about the reasoning process in the mind and then provides the User
    with the answer. The reasoning process is enclosed within <think> </think> and the answer is
    enclosed within <answer> </answer> tags, respectively, i.e., <think> reasoning process here </
    think> <answer> answer here </answer>.
User: {question-1}
Assistant: <think> {reasoning-1} </think> <answer> {answer-1} </answer>
User: {question-2}
Assistant: <think> {reasoning-2} </think> <answer> {answer-2} </answer>
User: {question-3}
Assistant: <think> {reasoning-3} </think> <answer> {answer-3} </answer>
User: {question}
Assistant: <think>
```

少样本提示通过给模型提供几个它要解决的任务示例来提升模型的性能。开放基础模型报告的基准测试数字往往是少样本的：例如，OLMo-2-0425-1B 模型卡上报告的 GSM8K 结果就使用了 8-shot 提示。我们在 cs336_alignment/prompts/r1_zero_three_shot_gsm8k.prompt 中包含了 r1_zero 提示的 3-shot 版本，其中的示例取自 OLMES 仓库。

最后，作为基线，我们将加入 question_only 提示，见 cs336_alignment/prompts/question_only.prompt ：

```
{question} Please put your final answer within \\boxed{{}}.
```

注意，虽然这个提示名叫 question_only ，我们仍要求把最终答案放进 \boxed{} 中，以便评分器更容易从回答中解析出答案，这将在下一节讨论。

3.3 评分函数

模型生成回答之后，我们需要检查它是否正确。我们的数学题带有标准答案（例如 0.5），但模型可能以多种方式给出正确答案：例如，它可以回答 <answer> 1/2 </answer> ，或者回答 The answer is 0.5. 。因此，为了恰当地给模型输出评分，我们需要一个答案解析函数，它以模型输出和一个已知的标准答案为输入，返回一个布尔值，表明模型输出是否正确。

在我们的实验中，我们将使用一个快速且相当准确的答案解析器，它来自最近关于推理 RL 的工作 [Z. Liu et al., 2025]。对于 r1_zero 提示，该奖励函数实现在 cs336_alignment.drgrpo_grader.r1_zero_reward_fn 。question_only 提示并不要求模型使用 <think> 和 <answer> 标签，因此应当改用同一个评分文件中的 cs336_alignment.drgrpo_grader.question_only_reward_fn 来

评估。注意，question_only 解析器会在 \boxed{} 中寻找最终答案。这些奖励函数返回一个总奖励（total reward），以及单独的格式奖励（format reward）和答案奖励（answer reward），分别表明回答是否符合预期格式、解析出的答案是否正确。一些工作会对格式正确但答案错误的回答给部分分数（即非零的总奖励）。在我们的实验中，我们不给部分分数，因此总奖励就等于答案奖励，格式奖励只用于日志记录。

注意，这些奖励函数期望 ground_truth 参数只包含答案。GSM8K 数据集中的标准答案形如 {rationale} #### {answer} ，要从中提取答案，你应当在 #### 处分割并去除空白字符。

3.4 实验

现在我们准备评估基础模型的提示性能了。

生成超参数。生成回答时，我们将以 temperature 1.0、top-p 1.0、最大生成长度 512 进行采样。r1_zero 系列提示要求模型以字符串 </answer> 结束它的答案，因此在使用这些提示时，我们可以指示 vLLM 在模型输出该字符串时停止：

```
# Based on Dr. GRPO: stop when the model completes its answer
# https://github.com/sail-sg/understand-r1-zero/blob/
#   c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params['stop'] = ["</answer>"]
sampling_params['include_stop_str_in_output'] = True
```

只对 r1_zero 和 r1_zero_three_shot 提示使用这个停止字符串，不要对 question_only 提示使用。

问题（prompting_baselines ）：在 GSM8K 上运行 OLMo-2-0425-1B（5 分）

(a) 编写一个脚本，用零样本 question_only 、零样本 r1_zero 和少样本 r1_zero_three_shot 三种提示评估 OLMo-2-0425-1B 在 GSM8K 上的性能。

然后，运行你的脚本并观察输出。对每种提示，模型生成中有多少落入以下各个类别：(1) 正确，且格式奖励与正确性奖励均为 1；(2) 格式奖励为 1、正确性奖励为 0；(3) 格式奖励为 0、正确性奖励为 0？观察至少十个类别 2 的例子，其中有多少模型输出实际上正确、只是没有被正确解析？类别 3 又如何？

交付要求：几句话的评论、评估指标，以及若干提示与回答的例子。

(b) 观察模型输出，刻画每种提示下模型的行为。例如，如果我们想让模型回答问题，只提供问题就够了吗？还是说模型在回答问题之外还表现出其他行为？零样本 r1_zero 和少样本 r1_zero_three_shot 提示是如何塑造模型行为的？

交付要求：几句话的评论，并附上支持性例子。

4 组相对策略优化

我们已经用纯提示的方式测量了模型性能，下一步将通过训练来提升性能。具体来说，我们想优化模型的准确率，或者等价地，它的期望奖励：

$$J_\theta = \mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_\theta(y|x)}[r(y \mid x)], \tag{1}$$

其中 ρ 是提示/问题 x 上的任务分布，π_θ 是我们的模型，y 是对 x 采样出的回答/解答，$r(y \mid x)$ 表示 y 是否是 x 的正确答案（我们也把 r 称为“奖励函数”）。注意，这个目标与课程到目前为止研究的预训练目标大不相同：交叉熵损失 $\mathbb{E}_{x\sim\mathcal{D}}[-\log \pi_\theta(x)]$ 从数据集 $\mathcal{D}$ 中抽取样本，而我们的准确率目标涉及对来自模型本身的样本求期望。因此，我们需要新的工具来优化这个目标：特别地，我们需要使用强化学习（RL）。从高层次来看，RL 包括：从模型采样回答、用评分函数给它们打分，并强化那些正确的回答。RL 通常又慢又充满挑战，所以本作业的很大一部分将探索让 RL 更快、更稳定的工具。

RL 的另一个方面是：由于它涉及从模型采样，其动态会受到采样时所使用提示的影响。具体而言，语言模型研究中一个令人兴奋的发现是：当使用思维链推理提示时，对一个强基础模型做 RL 可以大幅提升其推理能力 [OpenAI et al., 2024; DeepSeek-AI et al., 2025]。因此，在本作业中，除了探索 RL 的核心算法思想之外，我们还将探索 RL 与思维链提示等语言建模技术的相互作用。

4.1 推导在策略 GRPO

我们将首先探究 RL 背后的数学，一步步引出组相对策略优化（GRPO）算法——训练语言模型时常用的一种 RL 算法 [Z. Shao et al., 2024]。我们的讲述紧贴两份更深入讲解这些概念的优秀资料：OpenAI 的 *Spinning Up in Deep RL* [J. Achiam, 2018] 与 Nathan Lambert 的 *Reinforcement Learning from Human Feedback (RLHF) Book* [N. Lambert, 2024]。

4.1.1 将语言模型视为策略

在传统的强化学习中，我们考虑一个策略（policy），它接收状态（state）s_t 并输出动作（action）a_t。这个动作随后产生某个奖励 $r_t = r(s_t, a_t)$，以及一个从某个下一状态分布中抽出的后续状态 $s_{t+1} \sim \text{next_state}(s_t, a_t)$。强化学习就是优化策略以获得高奖励。

带参数 θ 的因果语言模型（LM）π_θ 定义了在给定当前文本前缀 $y_{<t} = (y_1, \ldots, y_{t-1})$ 的条件下下一个 token y_t 的概率分布。所以在 RL 的语境下，我们可以把下一个 token y_t 看作我们的动作 a_t，把当前文本前缀 $y_{<t}$ 看作状态 s_t。因此，LM 是一个类别型（categorical）随机策略

$$a_t \sim \pi_\theta(\cdot \mid s_t), \qquad \pi_\theta(a_t \mid s_t) = [\text{softmax}(f_\theta(s_t))]_{a_t}. \tag{2}$$

在用策略梯度优化策略时，我们需要两个基本操作：

1. 从策略采样：从上述类别分布中抽取一个动作 a_t；
2. 给动作的对数似然打分：计算 $\log \pi_\theta(a_t \mid s_t)$。

一般地，用 LLM 做 RL 时，s_t 是目前生成的部分补全/解答，每个 a_t 是解答的下一个 token；当模型输出一个文本结束（end-of-text）token 时，回合结束，比如 `<|end_of_text|>` ，或者在我们的 `r1_zero` 提示的情形下是 `</answer>` 。

4.1.2 轨迹

在强化学习中，我们从某个起始分布抽出的初始状态 $s_0 \sim \rho$ 出发。然后我们从策略 π_θ 采样一个动作 a_t，按照某个下一状态分布转移到下一个状态 $s_{t+1} \sim \text{next_state}(s_t, a_t)$，并不断重复。这一系列状态和动作就构成一条（有限视界的）轨迹（trajectory）：

$$\tau = (s_0, a_0, s_1, a_1, \ldots, s_T, a_T), \tag{3}$$

其中 T 是轨迹的长度，也就是说，a_T 是一个文本结束 token，或者我们已经达到按 token 计的最大生成预算。

在我们的情形中，$s_0 \sim \rho$ 就是包含我们想解的数学题的提示 x（可能附带了 CoT 提示或少样本示例）。从这个前缀出发，我们逐个采样 token，下一个状态是旧前缀拼接上新采样出的 token，即 $s_{t+1} = (s_t, a_t)$。轨迹也被称为回合（episode）或 rollout；我们将交替使用这些术语。

4.1.3 奖励与回报

在 RL 中，标量奖励 $r_t = r(s_t, a_t)$ 评判在状态 s_t 所采取动作的直接质量。对于可验证（verified）领域上的 RL，比如我们的数学应用题任务，中间推理步骤的奖励为零，直到我们输出答案。然后，我们在终止动作上收到一个经验证的奖励（verified reward）

$$r_T = r(s_T, a_T) := \begin{cases} 1 & \text{若轨迹 } \tau \text{ 依我们的奖励函数与标准答案吻合} \\ 0 & \text{否则} \end{cases} \tag{4}$$

在我们的情形中，如果最终答案正确，则 r_T 为 1，否则为 0。回报（return）$R(\tau)$ 把轨迹沿途的奖励汇总起来，在我们的情形中就是 r_T。

智能体的目标是最大化期望回报

$$J_\theta = \mathbb{E}_{\tau \sim \pi_\theta}[r(\tau)], \tag{5}$$

其中 $\tau \sim \pi_\theta$ 表示轨迹的分布：我们先采样 $s_0 \sim \rho$，然后从策略采样动作、从下一状态分布采样下一状态。在我们的情形中，这意味着先从数据集采样数学题 $x \sim \rho$，然后逐 token 采样直到到达回答末尾，这个过程可以记作 $y \sim \pi_\theta(y \mid x)$。该目标引出如下优化问题

$$\theta^* = \arg\max_\theta J_\theta. \tag{6}$$

4.1.4 策略梯度

现在我们有了学习一种称为策略梯度（policy gradients）的 RL 算法所需的记号和定义。在作业余下的部分，我们将坚持使用语言模型记号 (x, y) 来表示提示与回答，而不是状态 s_t 与动作 a_t 的 RL 记号。

回忆一下，我们想优化模型的期望奖励（即准确率），由下式给出

$$J_\theta = \mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_\theta(y|x)}[r(y \mid x)], \tag{7}$$

其中 $r(y \mid x)$ 表示回答 y 对提示 x 是否正确。我们的做法是对该目标做梯度上升，即

$$\theta_{k+1} = \theta_k + \alpha\nabla_\theta J_{\theta_k}, \tag{8}$$

但要做到这一点，我们需要能够从样本估计梯度 $\nabla_\theta J_{\theta_k}$。把这个梯度改写成对样本的期望，会得到一个被称为“REINFORCE 策略梯度”的表达式，其名称来自 REINFORCE 论文 [R. J. Williams, 1992]。具体而言，我们将看到可以把梯度改写为

$$\nabla_\theta\mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_\theta(y|x)}[r(y \mid x)] = \mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_\theta(y|x)}[r(y \mid x)\,\nabla_\theta \log \pi_\theta(y \mid x)]. \tag{9}$$

在走完推导之前，先注意这个表达式蕴含一个直观的算法（REINFORCE）：我们可以从数据集采样问题 x，从模型采样回答 y，在这些样本上计算 $r(y \mid x)\,\nabla_\theta \log \pi_\theta(y \mid x)$ 的平均，然后走一步梯度。表达式 $r(y \mid x)\,\nabla_\theta \log \pi_\theta(y \mid x)$ 的含义是：上调那些高奖励 $r(y \mid x)$ 的回答 y 的对数概率权重。如果我们重复这个过程并走多步梯度，就得到了一个基本的强化学习训练循环。

为了推导 REINFORCE 策略梯度，我们可以使用对数求导技巧（log-derivative trick；回忆 $\log f(x)$ 的导数是 $f'\frac{x}{f(x)}$）：

$$\nabla_\theta\pi_\theta(y \mid x) = \pi_\theta(y \mid x)\,\nabla_\theta \log \pi_\theta(y \mid x). \tag{10}$$

直接应用这个恒等式就得到想要的结果：

$$\begin{aligned}
\nabla_\theta\mathbb{E}_{y\sim\pi_\theta(y|x)}[r(y \mid x)] &= \sum_y \nabla_\theta\pi_\theta(y \mid x)\, r(y \mid x) && (11)\\
&= \sum_y \pi_\theta(y \mid x)\,\nabla_\theta \log \pi_\theta(y \mid x)\, r(y \mid x) && (12)\\
&= \mathbb{E}_{y\sim\pi_\theta(y|x)}[r(y \mid x)\,\nabla_\theta \log \pi_\theta(y \mid x)]. && (13)
\end{aligned}$$

注意，在这些等式中我们省略了对提示的期望 $\mathbb{E}_{x\sim\rho}$，因为 ∇_θ 与 $\mathbb{E}_{x\sim\rho}$ 可以交换。

于是，给定一批提示 $x^{(1)},\ldots,x^{(B)} \overset{\text{iid}}{\sim} \rho$，以及每个提示的 G 个回答 $y^{(i,j)} \overset{\text{iid}}{\sim} \pi_\theta(y \mid x^{(i)})$，我们有如下策略梯度的样本估计：

$$\hat{g} \leftarrow \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G} r(y^{(i,j)} \mid x^{(i)})\,\nabla_\theta \log \pi_\theta(y^{(i,j)} \mid x^{(i)}), \tag{14}$$

它上调高奖励回答的权重。为了推导 GRPO，我们将从这个基本策略梯度估计量出发，然后逐一施加修改。

4.1.5 基线

虽然基本的 REINFORCE 策略梯度估计量具有正确的期望 $\nabla_\theta J_\theta$，但它的方差很大，使 RL 不稳定。为了稳定训练并缩减方差，人们使用的一种工具是基线（baseline）。基线可以做得非常复杂，而最简单的基线是从奖励中减去一个常数 b：

$$\mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_\theta(y|x)}[(r(y \mid x) - b)\,\nabla_\theta \log \pi_\theta(y \mid x)]. \tag{15}$$

例如，如果我们减去 $b = 0.5$，而奖励 r 是二值的（正确为 1，错误为 0），使用这个估计量意味着：我们从模型采样回答，上调正确回答的权重（权重为 0.5），并下调错误回答的权重（权重为 -0.5）。这与我们最初的梯度估计量形成对比：后者以权重 1 上调正确回答的权重，而对错误回答不做任何事（权重为 0）。

本节关键的数学结论告诉我们：只要基线 b 不依赖于动作/回答 y（例如它可以是常数、x 的函数，或策略的函数），减去基线 b 就能保持估计量的期望。只要 b 不依赖于 y，在估计量中减去它就能保持其期望：

$$\mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_\theta(y|x)}[(r(y \mid x) - b)\,\nabla_\theta \log \pi_\theta(y \mid x)] = \mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_\theta(y|x)}[r(y \mid x)\,\nabla_\theta \log \pi_\theta(y \mid x)]. \tag{16}$$

这个事实直接来自如下恒等式：

$$\mathbb{E}_{y\sim\pi_\theta(y|x)}[\nabla_\theta \log \pi_\theta(y \mid x)] = \nabla_\theta \mathbb{E}_{y\sim\pi_\theta(y|x)}[1] = 0, \tag{17}$$

其中第一步来自把之前的对数求导技巧反向使用。

虽然基线保持估计量的期望，但它们可能增大或减小方差。用记号来表述，“方差”的含义如下：把我们的策略梯度估计量写成样本均值 $\frac{1}{n}\sum_{i=1}^{n} Z_i$，其中每个 Z_i 是对样本 (x, y) 取的表达式 $(r(y \mid x) - b)\nabla_\theta \log \pi_\theta(y \mid x)$，则估计量的方差是 $\frac{\mathbb{E}[Z_i^2]-\mathbb{E}[Z_i]^2}{n}$。我们将在下面的问题中探讨基线何时增大或减小方差。

问题（`baseline_calcs`）：计算策略梯度估计量的方差（5 分）

设 π_θ 定义了二值动作空间 $\mathcal{A} = \{0, 1\}$ 上的一个策略，$\pi_\theta(A{=}1) = p = \sigma(\theta)$，其中 σ 表示 sigmoid 函数 $\sigma(\theta) = \frac{1}{1+e^{-\theta}}$。我们的二值奖励函数给动作 $A = 1$ 赋予奖励 1，否则为 0，即 $r(A) = \mathbf{1}\{A = 1\}$。

(a) 设策略梯度估计量为

$$\frac{1}{n}\sum_{i=1}^{n} r(A_i)\,\nabla_\theta \log \pi_\theta(A_i) \tag{18}$$

给定 n 个样本 $A_i \overset{\text{iid}}{\sim} \pi_\theta$。这个估计量的方差是多少？

交付要求：一个用 n 和 p 表示的表达式，并附上推导。

(b) 设经基线调整的策略梯度估计量为

$$\frac{1}{n}\sum_{i=1}^{n} (r(A_i) - b)\,\nabla_\theta \log \pi_\theta(A_i) \tag{19}$$

给定 n 个样本 $A_i \overset{\text{iid}}{\sim} \pi_\theta$。这个估计量的方差是多少？

交付要求：一个用 n、b 和 p 表示的表达式，并附上推导和一些讨论。

(c) 如果我们代入“总体均值”基线 $b = p$，得到的方差是多少？把这个方差与未调整的策略梯度估计量的方差比较：它是总是更低、总是更高，还是视 p 而定有时更高有时更低？

GRPO 中的基线

GRPO 的第一个组件是应用“组均值”基线：

$$\hat{g} \leftarrow \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\Big(r(y^{(i,j)} \mid x^{(i)}) - \mu_i\Big)\,\nabla_\theta \log \pi_\theta(y^{(i,j)} \mid x^{(i)}) \tag{20}$$

其中 $\mu_i = \frac{1}{G}\sum_{j=1}^{G} r(y^{(i,j)} \mid x^{(i)})$，即模型在提示 $x^{(i)}$ 上的平均奖励。注意，尽管 μ_i 依赖于回答 y，它仍在乘以缩放因子 $\frac{G-1}{G}$ 的意义上保持期望：

$$\mathbb{E}_{y^{(1)},\ldots,y^{(G)}\sim\pi_\theta(y|x)}\left[\frac{1}{G}\sum_{j=1}^{G}\Big(r(y^{(j)} \mid x) - \mu_i\Big)\,\nabla_\theta \log \pi_\theta(y^{(j)} \mid x)\right] \tag{21}$$

$$= \frac{1}{G}\sum_{j=1}^{G}\mathbb{E}_{y^{(2)},\ldots,y^{(G)}\sim\pi_\theta(y|x)}\left[\mathbb{E}_{y^{(1)}\sim\pi_\theta(y|x)}\left[\Big(r(y^{(1)} \mid x) - \mu_i\Big)\,\nabla_\theta \log \pi_\theta(y^{(1)} \mid x)\right]\right] \tag{22}$$

$$= \frac{1}{G}\sum_{j=1}^{G}\mathbb{E}_{y^{(1)}\sim\pi_\theta(y|x)}\left[\left(r(y^{(1)} \mid x) - \frac{1}{G}r(y^{(1)} \mid x)\right)\nabla_\theta \log \pi_\theta(y^{(1)} \mid x)\right] \tag{23}$$

$$= \frac{G-1}{G}\,\mathbb{E}_{y\sim\pi_\theta(y|x)}[r(y \mid x)\,\nabla_\theta \log \pi_\theta(y \mid x)]. \tag{24}$$

这些经过组均值调整的奖励常被称为优势（advantage）：这一项表示的不是绝对奖励，而是该 rollout 相对于平均奖励的“优势”。

4.1.6 优势归一化

GRPO 在减去组均值之后的下一个修改，是除以组标准差：

$$\hat{g} \leftarrow \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{r(y^{(i,j)} \mid x^{(i)}) - \mu_i}{\mathrm{std}_i}\,\nabla_\theta \log \pi_\theta(y^{(i,j)} \mid x^{(i)}), \tag{25}$$

其中 $\mathrm{std}_i = \sqrt{\frac{1}{G}\sum_{j=1}^{G}\big(r(y^{(i,j)} \mid x^{(i)}) - \mu_i\big)^2}$（注意，默认的 `torch.std` 函数使用一个略有不同的样本标准差表达式来做偏差校正；你应当在实现中使用默认的 `torch.std` ）。除以标准差不再保持估计量的期望，所以我们不再对原目标 J_θ（期望奖励）做梯度上升。可以把这个修改理解为一种稳定性技巧：如果我们假设每个梯度向量都是 iid 高斯的，

那么除以组标准差就能保证每一组的梯度更新在范数上大致相等。我们将在下一节更深入地讨论优势归一化。

4.1.7 序列归一化

GRPO 中还有另一个似乎是从早期 LLM-PPO 实现继承而来的细节，称为序列归一化（sequence normalization）。回忆一下，我们（到目前为止）的策略梯度估计量如下（其中我们把 $\log \pi_\theta(y^{(i,j)} \mid x^{(i)})$ 展开成了对时间步的求和）：

$$\hat{g} \leftarrow \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{r(y^{(i,j)} \mid x^{(i)}) - \mu_i}{\mathrm{std}_i}\,\nabla_\theta \sum_{t=1}^{\mathrm{len}(y^{(i,j)})}\log \pi_\theta\Big(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\Big). \tag{26}$$

GRPO 又增加了一个序列长度归一化项：

$$\hat{g} \leftarrow \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{r(y^{(i,j)} \mid x^{(i)}) - \mu_i}{\mathrm{std}_i}\nabla_\theta\left(\frac{1}{\mathrm{len}(y^{(i,j)})}\sum_{t=1}^{\mathrm{len}(y^{(i,j)})}\log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right)\right). \tag{27}$$

注意，这个修改进一步改变了期望：它不再对跨序列的所有 token 一视同仁，较长序列中的 token 相对较短序列中的 token 被降权。在下一节中，我们会讨论这个选择，并通过消融实验来看它是否是一个好的修改。但现在，我们只实现原论文中提出的标准 GRPO 算法，所以我们会包含序列归一化。

4.1.8 整合所有组件

GRPO 还有一个我们尚未讨论的组件，即裁剪的重要性重加权（clipped importance reweighting）。我们将在作业后面讨论离策略 RL 时再介绍这个组件。简要说来，离策略 RL 指的是通过在每个推理批次上走多个梯度步来加速 RL 的做法；与之相对，在策略 RL 在每个推理批次上只走一个梯度步。“离策略”一词指的是：从第二个梯度步开始，推理批次中的样本已经过时（stale），因为它们来自模型的旧版本。重要性重加权就是对梯度估计量的一种修改，使它能够使用过时样本。眼下我们只做在策略 RL，所以还不需要重要性重加权。

现在，我们拥有了实现在策略 GRPO 所需的全部组件。算法见算法 1。

算法 1：在策略组相对策略优化（GRPO）

输入 初始策略模型 π_{θ_0}；奖励函数 r；任务分布（或数据集）ρ；学习率 α

输出 π_θ

1. 策略模型 $\pi_\theta \leftarrow \pi_{\theta_0}$
2. 对 `step` $= 1, \ldots,$ `n_grpo_steps` :
3. 采样一批问题 $x^{(1)}, \ldots, x^{(B)} \overset{\text{iid}}{\sim} \rho$
4. 为每个问题采样 G 个输出：$y^{(i,1)}, \ldots, y^{(i,G)} \overset{\text{iid}}{\sim} \pi_\theta(y \mid x^{(i)})$
5. 计算在策略 GRPO 策略梯度估计量

$$\hat{g} \leftarrow \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\mathrm{len}(y^{(i,j)})}\sum_{t=1}^{\mathrm{len}(y^{(i,j)})}\frac{r(y^{(i,j)} \mid x^{(i)}) - \mu_i}{\mathrm{std}_i}\nabla_\theta \log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right), \tag{28}$$

其中

$$\mu_i = \frac{1}{G}\sum_{j=1}^{G} r(y^{(i,j)} \mid x^{(i)}) \tag{29}$$

表示组均值，而

$$\mathrm{std}_i = \sqrt{\frac{1}{G}\sum_{j=1}^{G}\left(r(y^{(i,j)} \mid x^{(i)}) - \mu_i\right)^2} \tag{30}$$

表示组标准差。

6. 用你喜欢的优化器更新 θ（我们在下面给出随机梯度上升更新）：

$$\theta \leftarrow \theta + \alpha \hat{g}. \tag{31}$$

4.2 实现在策略 GRPO

4.2.1 使用 Hugging Face 模型

虽然前几次作业使用的是我们自己在 `cs336_basics` 中的语言模型实现，但本作业将直接使用 Hugging Face `transformers` 库来加载预训练的基础模型。如果你愿意，也欢迎自己实现 transformer 和预训练权重加载器；只要确保架构与 OLMo-2-0425-1B 的一致即可。

要加载 Hugging Face 模型和分词器（以 bfloat16 并使用 FlashAttention-2 以节省内存），可以使用以下起始代码，见 `cs336_alignment/checkpoint.py` ：

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

def get_model_and_tokenizer(model_id_or_dir: str, device: str):
    model = AutoModelForCausalLM.from_pretrained(
        model_id_or_dir,
        device_map=device,
        torch_dtype=torch.bfloat16,
        attn_implementation="eager" if device=='cpu' else "flash_attention_2",
    )
```

```
    tokenizer = AutoTokenizer.from_pretrained(model_id_or_dir)
    return model, tokenizer
```

`model_id_or_directory` 可以是类似 `allenai/OLMo-2-0425-1B` 的模型名称，也可以是一个目录路径。该目录通常就是调用 `save_pretrained` 的结果——你可以调用它来保存自己训练的模型：

```
# Save the model weights
model.save_pretrained(save_directory=output_dir)
tokenizer.save_pretrained(save_directory=output_dir)
```

首先，让我们实现一个辅助函数，它使用预训练的分词器对输入的提示（prompt）和回答（response）进行分词。请分别对提示和回答分词、不添加任何特殊 token，然后把它们直接拼接为 `prompt_ids + response_ids` ，中间不插入 EOS、BOS 或任何分隔 token。我们还需要构造一个 `response_mask` ：一个布尔掩码，与右移后的 `labels` 对齐，在标签 token 来自回答的位置恰好为 `True` ，在提示与填充（padding）位置为 `False` 。我们将在训练循环中使用这个掩码，以确保只对回答部分的 token 计算损失。

问题（`tokenize_prompt_and_output` ）：提示与输出的分词（1 分）

交付要求：实现一个方法 `tokenize_prompt_and_output` ，它分别对提示和输出分词、在不插入特殊 token 的情况下把它们拼接在一起，并构造出 `response_mask` 。推荐使用如下接口：

```
def tokenize_prompt_and_output(
       prompt_strs: list[str],
       output_strs: list[str],
       tokenizer: PreTrainedTokenizer,

) -> dict[str, torch.Tensor]:
```

对提示和输出字符串分词，并构造一个与 `labels` 对齐的掩码：回答 token 对应的位置为 1，其他位置（提示或填充）为 0。

参数：

- `prompt_strs: list[str]` 提示字符串的列表。
- `output_strs: list[str]` 输出字符串的列表。
- `tokenizer: PreTrainedTokenizer` 用于分词的分词器。

返回值：

- `dict[str, torch.Tensor]` 。设 `prompt_and_output_lens` 为一个列表，包含拼接后的提示与输出字符串各自的长度。那么返回的字典应包含以下键：
  - `input_ids` :形状为 `(batch_size, max(prompt_and_output_lens) - 1)` 的 `torch.Tensor` ：分词后的提示与输出字符串，切掉了最后一个 token。
  - `labels` ：形状为 `(batch_size, max(prompt_and_output_lens) - 1)` 的 `torch.Tensor` ：右移后的 input ids，即去掉了第一个 token 的 input ids。
  - `response_mask` ：形状为 `(batch_size, max(prompt_and_output_lens) - 1)` 的 `torch.Tensor` ：与 `labels` 对齐的掩码，当对应的标签 token 属于回答时取值为 1，否则为 0。

要测试你的代码，请实现 `adapters.run_tokenize_prompt_and_output` 。然后用 `uv run pytest -k test_tokenize_prompt_and_output` 运行测试，确保你的实现通过测试。

拿到分词后的输入之后，我们可以把它传入模型：

```
input_ids = train_batch["input_ids"].to(device)
labels = train_batch["labels"].to(device)
logits = model(input_ids).logits
```

掌握这一语法后，请实现下面这个方法：它计算模型下回答中每个 token 的对数概率（log-probability）——这是计算策略梯度所需要的基本量。在强化学习（RL）中，记录每个 token 的熵（entropy）也往往很有用，因此该函数还包含一个需要你一并实现的 `return_token_entropy` 选项。

问题（`get_response_log_probs` ）：响应的对数概率（与熵）（1 分）

交付要求：实现一个方法 `get_response_log_probs` ，它从因果语言模型中获取每个 token 的条件对数概率（在给定前面 token 的条件下），并可选地返回模型下一 token 分布的熵。

推荐使用如下接口：

```
def get_response_log_probs(
       model: PreTrainedModel,
       input_ids: torch.Tensor,
       labels: torch.Tensor,
       return_token_entropy: bool = False,

) -> dict[str, torch.Tensor]:
```

参数：

- `model: PreTrainedModel` 用于打分的 HuggingFace 模型（应放在正确的设备上；如果不需要计算梯度，则处于推理模式）。
- `input_ids: torch.Tensor` 形状为 `(batch_size, sequence_length)`，由你的分词方法产生的提示 + 回答拼接后的 token。
- `labels: torch.Tensor` 形状为 `(batch_size, sequence_length)`，由你的分词方法产生的 labels。
- `return_token_entropy: bool` 若为 `True`，则同时返回每个 token 的熵。

返回值：

- `dict[str, torch.Tensor]` 。
  - `"log_probs"` 形状为 `(batch_size, sequence_length)`，条件对数概率 $\log p_\theta(x_t \mid x_{<t})$。
  - `"token_entropy"` 可选，形状为 `(batch_size, sequence_length)`，每个位置上 token 的熵（仅当 `return_token_entropy=True` 时存在）。

要测试你的代码，请实现 `adapters.run_get_response_log_probs` 。然后运行 `uv run pytest -k test_get_response_log_probs`，确保测试通过。

4.2.2 在强化学习循环中使用 vLLM

在 RL 循环中，我们还需要生成 rollout（即模型对提示的采样回答）。我们将采用的具体配置是：把 Hugging Face 模型和优化器放在一块 GPU 上用于训练，把 vLLM（包含模型和 KV 缓存）放在另一块 GPU 上。因此，除了上一节描述的 vLLM 初始化与生成函数之外，我们还需要在每次推理步骤之前在两个设备之间同步权重。权重量同步代码如下，位于 `cs336_alignment/vllm_utils.py` 。

```
@dataclass
class VLLMServer:
    gpu: int = 1 # Run training on gpu 0 and inference on gpu 1
```

```
    # Create the NCCL weight-transfer group between the training GPU and vLLM.
    def init_weight_sync(self, policy_device: str): ...

    # Copy the current Hugging Face policy weights into the vLLM server and
    # reset vLLM caches that depended on the old weights.
    def sync_policy_weights(self, policy: torch.nn.Module) -> None: ...

    # Generate rollouts from the current vLLM weights.
    def generate_completions(
        self,
        prompts: list[str],
        sampling_params: dict,
        batch_size: int | None = None,
    ) -> list[VLLMCompletion]: ...
```

与之前的提示实验一样，我们使用温度 1.0、top-p 1.0、最大生成长度 512 进行采样。提示词要求模型以字符串 `</answer>` 结束它的回答，因此我们可以指示 vLLM 在模型输出该字符串时停止生成：

```
# Based on Dr. GRPO: stop when the model completes its answer
# https://github.com/sail-sg/understand-r1-zero/blob/
#   c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params['stop'] = ["</answer>"]
sampling_params['include_stop_str_in_output'] = True
```

4.2.3 GRPO 组件

接下来，让我们实现用于计算 GRPO 损失各个部分的一些组件。注意，在作业的后面部分，我们会实现各种变体，比如不同的优势（advantage）归一化方法或重要性重加权（importance reweighting）方法。下面的组件在设计上允许我们以最少的代码重复在各变体之间切换，因此我们提供的接口带有用于指定变体的参数。在作业的这一部分，你只需要实现标准 GRPO 变体。

我们的第一步是实现一个辅助函数，用于计算各个回答的奖励。

问题（`compute_rollout_rewards`）：计算 rollout 的奖励（1 分）

交付要求：实现一个方法 `compute_rollout_rewards`，为每条 rollout 回答计算原始奖励。

推荐使用如下接口：

```
def compute_rollout_rewards(
       reward_fn: Callable[[str, str], dict[str, float]],
       rollout_responses: list[str],
       repeated_ground_truths: list[str],

) -> tuple[torch.Tensor, dict[str, float]]:
```

为一组 rollout 回答计算奖励，并返回奖励各组成部分的元数据（metadata）。

参数：

- `reward_fn: Callable[[str, str], dict[str, float]]` 将 rollout 回答与标准答案对比打分，产生一个包含键 `"reward"` 、`"format_reward"` 和 `"answer_reward"` 的字典。
- `rollout_responses: list[str]` 来自策略的 rollout。该列表的长度为 `rollout_batch_size = n_prompts_per_rollout_batch * group_size` 。
- `repeated_ground_truths: list[str]` 各样本的标准答案。该列表的长度为 `rollout_batch_size` ，因为每个样本的标准答案会重复 `group_size` 次。

返回值：

- `tuple[torch.Tensor, dict[str, float]]` 。
  - `raw_rewards` 形状为 `(rollout_batch_size,)` 。每条 rollout 回答的未归一化奖励。
  - `metadata` 要记录的奖励统计量。至少要包含整个 rollout 批次上总奖励和格式奖励的均值。

要测试你的代码，请实现 `adapters.run_compute_rollout_rewards` 。然后用 `uv run pytest -k compute_rollout_rewards` 运行测试，确保你的实现通过测试。

接下来，让我们实现 GRPO 中的核心数学：将这些奖励归一化，从而把它们转化为优势。注意，上面函数输出的奖励是被拉平（flatten）的，因此我们需要传入组大小（group size），把它重新整理成一个个组再做归一化。为了通过测试，你需要支持 `baseline = "mean"` 搭配 `advantage_normalizer = "std"` 。为避免除以零，你还应在做除法之前把 `advantage_eps` 加到每组的标准差上。

问题（`compute_group_normalized_rewards_grpo`）：组归一化（1 分）

交付要求：实现一个方法 `compute_group_normalized_rewards` ，它在各组内部对原始奖励做归一化，并返回归一化后的奖励以及任何你认为有用的元数据。

目前你只需要支持 `baseline = "mean"` 和 `advantage_normalizer = "std"` 。对于不支持输入，直接抛出 `NotImplementedError` 即可。在作业后面的部分，我们会实现其他选项并进行消融实验。记住要在归一化分母上加上 `advantage_eps` ，以避免除以零。

推荐使用如下接口：

```
def compute_group_normalized_rewards(
       raw_rewards: torch.Tensor,
       group_size: int,
       baseline: Literal["mean", "none"] = "mean",
       advantage_eps: float = 1e-6,
       advantage_normalizer: Literal["std", "none", "mean"] = "std",

):
```

通过在每组内部应用指定的基线（baseline）和归一化方式来计算优势。

参数：

- `raw_rewards: torch.Tensor` 形状为 `(rollout_batch_size,)` 。每条 rollout 回答的未归一化奖励，其中 `rollout_batch_size = n_prompts_per_rollout_batch * group_size` 。
- `group_size: int` 每个问题（组）对应的回答数量。
- `baseline: Literal["mean", "none"]` 本题只需支持 `mean` ，即减去组内平均奖励。之后 `none` 将表示不做基线扣减。
- `advantage_eps: float` 一个小常数，用于避免归一化时除以零。
- `advantage_normalizer: Literal["std", "none", "mean"]` 本题只需支持 `std` ，即除以组内标准差。之后 `none` 将表示不做归一化，`mean` 将表示除以组内平均奖励。

返回值：

- `tuple[torch.Tensor, dict[str, float]]` 。
  - `advantages` 形状为 `(rollout_batch_size,)` 。每条 rollout 回答的组内归一化奖励。
  - `metadata` 由你自行选择要记录的其他统计量（例如奖励的均值、标准差、最大/最小值）。

要测试你的代码，请实现 `adapters.run_compute_group_normalized_rewards` 。然后用 `uv run pytest -k compute_group_normalized_rewards_grpo` 运行测试，确保你的实现通过测试。

下一步是实现一个逐 token 计算策略梯度损失的函数。注意：算法 1 中的公式写出的是梯度估计量，而本题要求你写出一个逐 token 的损失（loss），使得对它求梯度后能得到 GRPO 梯度估计量的每一项（随后我们会在 token 维度上聚合）。这个表达式并不是传统意义上期望在训练中不断下降的损失；它只是一个表达式，使得对它取梯度时恰好得到策略梯度。

你要实现的具体损失是

$$J_\theta^{\text{GRPO-on-policy}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\operatorname{len}\left(y^{(i,j)}\right)}\sum_{t=1}^{\operatorname{len}\left(y^{(i,j)}\right)}\frac{r\left(y^{(i,j)} \mid x^{(i)}\right)-\mu_i}{\operatorname{std}_i}\log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right), \tag{32}$$

你可以验证，对它求微分恰好产生算法 1 中的梯度项。由于 PyTorch 优化器执行的是梯度下降，你的实现应返回该目标的相反数作为损失。`compute_policy_gradient_loss` 负责逐 token、逐序列地计算损失项，而 `aggregate_loss_across_microbatch_sequence` 负责在 token 和序列维度上对逐 token 损失取平均。

问题（`compute_policy_gradient_loss_on_policy` ）：在策略策略梯度（1 分）

交付要求：实现一个方法 `compute_policy_gradient_loss` ，在给定原始奖励或预先计算好的优势的情况下，计算逐 token 的策略梯度损失。

目前我们假设所有 rollout 都是在策略（on-policy）的，因此你只需要支持 `importance_reweighting_method = "none"` ，并且可以忽略旧对数概率（old-log-prob）和裁剪（clipping）相关的参数。对于不支持的输入，直接抛出 `NotImplementedError` 即可。我们会在作业后面实现裁剪。

推荐使用如下接口：

```
def compute_policy_gradient_loss(
       raw_rewards_or_advantages: torch.Tensor,
       policy_log_probs: torch.Tensor,
       importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"] = "none",
       old_log_probs: torch.Tensor | None = None,
       cliprange: float | None = None,
       response_mask: torch.Tensor | None = None,

) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

在每个 token 处计算策略梯度损失，其中 `raw_rewards_or_advantages` 既可以是原始奖励，也可以是已经归一化的优势。

参数：

- `raw_rewards_or_advantages: torch.Tensor` 形状为 `(batch_size,)` 或 `(batch_size, 1)` ,每条 rollout 回答的标量奖励/优势。
- `policy_log_probs: torch.Tensor` 形状为 `(batch_size, sequence_length)` ，每个 token 的对数概率。
- `importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"]` `"none"` ：不做重要性重加权；`"noclip"` ：应用重要性重加权但不裁剪；`"grpo"` ：进行 PPO/GRPO 风格的 token 级重加权与裁剪；`"gspo"` ：进行 GSPO 风格的序列级重加权与裁剪。
- `old_log_probs: torch.Tensor | None` 除 `importance_reweighting_method = "none"` 外必填；形状为 `(batch_size, sequence_length)` 。
- `cliprange: float | None = None` 裁剪参数 ε，当 `importance_reweighting_method` 为 `"grpo"` 或 `"gspo"` 时必填。
- `response_mask: torch.Tensor | None = None` 可选,形状为 `(batch_size, sequence_length)` 的回答 token 掩码。对于只在回答 token 上平均序列级对数比率的 GSPO 实现来说是必需的。

返回值：

- `tuple[torch.Tensor, dict[str, torch.Tensor]]` 。
  - `per_token_policy_gradient_loss` 形状为 `(batch_size, sequence_length)` ，逐 token 的策略梯度损失（将在训练循环中沿 batch 和 sequence 维度聚合）。
  - `metadata` 来自底层损失调用的统计量，例如裁剪比例（clip fraction）的相关分量。

要测试你的代码，请实现 `adapters.run_compute_policy_gradient_loss` 。然后运行 `uv run pytest -k test_compute_policy_gradient_loss_on_policy` ，确保测试通过。

最后，让我们实现跨 token 和跨序列的聚合函数：标准 GRPO 的做法是先对每条序列内的 token 取平均，再对序列取平均。稍后我们会实现另一种做法——直接除以一个常数，正如推导中所见，这种做法更忠实于我们最初想要估计的策略梯度。

问题（aggregate_loss_across_microbatch_sequence ）：跨 token 与序列聚合损失（0.5 分）

交付要求：实现一个方法 aggregate_loss_across_microbatch ，它接收逐 token 的策略梯度损失和回答掩码，输出平均损失。目前我们使用标准 GRPO 聚合方式，即先在每条序列内部取平均、再对序列取平均，因此你可以假设 loss_normalization = "sequence" 。对于不支持的输入，直接抛出 NotImplementedError 即可。

推荐使用如下接口：

```
def aggregate_loss_across_microbatch(
       per_token_policy_gradient_loss: torch.Tensor,
       mask: torch.Tensor,
       loss_normalization: Literal["sequence", "constant"] = "sequence",
       normalization_constant: int | None = None,

) -> torch.Tensor:
```

根据回答掩码和损失归一化策略，聚合逐 token 的策略梯度损失。

参数：

- per_token_policy_gradient_loss: torch.Tensor 形状为 (batch_size, sequence_length) ，逐 token 的策略梯度损失（将在训练循环中沿 batch 和 sequence 维度聚合）。
- mask 形状为 (batch_size, sequence_length) 的 torch.Tensor ，指示哪些位置应计入损失。
- loss_normalization: Literal["sequence", "constant"] = "sequence" "sequence" ：先对每条序列内的损失取平均，再对序列取平均；"constant" ：用一个常数对总损失归一化。
- normalization_constant: int | None = None 用于除总损失的常数；当 loss_normalization = "constant" 时必填。

返回值：

- loss: torch.Tensor 包含平均损失的标量。请确保之后可以对这个损失调用 backward 。

要测试你的代码，请实现 adapters.run_aggregate_loss_across_microbatch 。然后运行 uv run pytest -k test_aggregate_loss_across_microbatch_sequence ，确保测试通过。

4.2.4 GRPO 训练步

现在我们准备好把这些组件拼装成一个完整的训练步了。

梯度累积

为了在推理时获得良好的利用率，我们需要较大的批量大小（batch size）。于是，在训练批量大小等于推理批量大小的在策略 RL 中，GPU 没有足够的显存一次性计算整个批次的梯度。因此，我们需要把批次切分成一系列微批次（microbatch），并在这些微批次之间累积梯度。其中最主要的麻烦在于正确处理归一化，以保证微批次累积得到的梯度等价于在整个批次上计算的梯度。

梯度累积在 PyTorch 中很容易实现。回忆一下，每个权重张量都有一个存储其梯度的属性 `.grad` 。在调用 `loss.backward()` 之前，`.grad` 属性为 `None` ；调用 `loss.backward()` 之后，`.grad` 属性便包含梯度。通常我们会走一步优化器，然后用 `optimizer.zero_grad()` 把梯度清零，这会重置权重张量的 `.grad` 字段：

```
# Forward pass.
logits = model(inputs)
loss = loss_fn(logits, labels)

# Backward pass.
loss.backward()

# Update weights.
optimizer.step()
# Zero gradients in preparation for next iteration.
optimizer.zero_grad()
```

要实现梯度累积，我们把批次切分成 k 个微批次，并且只调用一次 `optimizer.step()` 和 `optimizer.zero_grad()` 。假设采用序列归一化（先对每条序列的损失取平均、再对序列取平均），那么在计算出微批次的平均损失之后，我们还需要按序列数量重新加权。

```
gradient_accumulation_steps = 4
microbatch_size = len(inputs) // gradient_accumulation_steps
for i in range(0, len(inputs), microbatch_size):
    inputs_microbatch = inputs[i:i+microbatch_size]
    labels_microbatch = labels[i:i+microbatch_size]

    # Forward pass.
    logits = model(inputs_microbatch)
    loss = loss_fn(logits, labels_microbatch) * (len(inputs_microbatch) / len(inputs))

    # Backward pass.
    loss.backward()
# Update weights once across entire batch.
optimizer.step()
# Zero gradients once across entire batch.
optimizer.zero_grad()
```

注意，稍后当我们改用常数归一化而不是序列归一化时，微批次代码会稍有变化，以适应不同的归一化方式。

实现训练步

在下面的函数中，我们终于要实现一个 GRPO 训练步，输入是某一批 rollout。该函数接收模

型、分词器、优化器、奖励函数、提示与 rollout 以及各种超参数，并负责累积梯度、走一步优化器。你需要按照上面的描述实现梯度累积，以免显存耗尽。在优化器步之前，函数还应把梯度范数裁剪到 `max_grad_norm` 。随后函数应返回该批次上的训练损失以及元数据，两者都用于日志记录。请在训练步中至少记录以下指标：

- 损失。
- 梯度范数。
- token 熵。
- 训练奖励（总奖励、格式奖励）。

你也欢迎记录其他指标：例如，记录 pass@k（k 介于 1 和 `group_size` 之间，即前 k 条 rollout 中至少有一条正确回答的提示所占的比例）可能会很有用。

问题（`grpo_train_step_standard_on_policy` ）：GRPO 训练步（5 分）

交付要求：在给定模型、分词器和 rollout 的情况下，实现一次策略梯度的批量更新。在作业的这一部分，你只需要针对在策略设定下的标准 GRPO 实现该函数，即 `baseline = "mean"` 、`advantage_normalizer = "std"` 、`importance_reweighting_method = "none"` 、`loss_normalization = "sequence"` 。对于不支持的输入，直接抛出 `NotImplementedError` 即可。

推荐使用如下接口：

```
def grpo_train_step(
       model: PreTrainedModel,
       tokenizer: PreTrainedTokenizer,
       optimizer: Optimizer,
       gradient_accumulation_steps: int,
       max_grad_norm: float | None,
       reward_fn: Callable[[str, str], dict[str, float]],
       repeated_prompts: list[str],
       rollout_responses: list[str],
       repeated_ground_truths: list[str],
       group_size: int,
       # Reward normalization
       baseline: Literal["mean", "none"] = "mean",
       advantage_eps: float = 1e-6,
       advantage_normalizer: Literal["std", "none", "mean"] = "std",
       # Importance reweighting and clipping
       importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"] = "none",
       old_log_probs: torch.Tensor | None = None,
       cliprange: float | None = None,
       # Loss normalization
       loss_normalization: Literal["sequence", "constant"] = "sequence",
       normalization_constant: int | None = None,

) -> tuple[torch.Tensor, dict[str, torch.Tensor | float]]:
```

使用 gradient_accumulation_steps 个微批次执行前向与反向传播。

参数：

- model: PreTrainedModel 要训练的 HuggingFace 模型。
- tokenizer: PreTrainedTokenizer 用于分词的分词器。
- optimizer: Optimizer 模型的优化器。
- gradient_accumulation_steps: int 每次优化器步包含的微批次数量。
- max_grad_norm: float | None 若不为 None ，则在调用 optimizer.step() 之前把梯度范数裁剪到该值。
- reward_fn: Callable[[str, str], dict[str, float]] 将 rollout 回答与标准答案对比打分，产生一个包含键 "reward" 、"format_reward" 和 "answer_reward" 的字典。
- repeated_prompts: list[str] 各样本的提示。该列表的长度为 rollout_batch_size ，因为每个样本的提示会重复 group_size 次。
- rollout_responses: list[str] 来自策略的 rollout。该列表的长度为 rollout_batch_size = n_prompts_per_rollout_batch * group_size 。
- repeated_ground_truths: list[str] 各样本的标准答案。该列表的长度为 rollout_batch_size ，因为每个样本的标准答案会重复 group_size 次。
- group_size: int 每个问题（组）对应的回答数量。
- baseline: Literal["mean", "none"] 若为 mean ，减去组内平均奖励；若为 none ，则不做任何处理。
- advantage_eps: float 一个小常数，用于避免归一化时除以零。
- advantage_normalizer: Literal["std", "none", "mean"] 若为 std ，除以组内标准差；若为 none ，则不做任何处理；若为 mean ，除以组内平均奖励。
- importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"] "none" ：不做重要性重加权；"noclip" ：应用重要性重加权但不裁剪；"grpo" ：进行 PPO/GRPO 风格的 token 级重加权与裁剪；"gspo" ：进行 GSPO 风格的序列级重加权与裁剪。
- old_log_probs: torch.Tensor | None 除 importance_reweighting_method = "none" 外必填；形状为 (batch_size, sequence_length) 。
- cliprange: float | None = None 裁剪参数 ε，当 importance_reweighting_method 为 "grpo" 或 "gspo" 时必填。
- loss_normalization: Literal["sequence", "constant"] = "sequence" "sequence" ：先对每条序列内的损失取平均，再对序列取平均；"constant" ：用一个常数对总损失归一化（在整个训练过程中固定不变）。

- `normalization_constant: int | None = None` 用于除总损失的常数；当 `loss_normalization = "constant"` 时必填。

返回值：

- `tuple[torch.Tensor, dict[str, torch.Tensor]]` 。
  - `loss` 标量张量。经梯度累积调整后的批次损失。我们返回它以便记录日志。
  - `metadata` 字典，包含来自底层损失调用的元数据、裁剪前的梯度范数，以及你可能想记录的其他统计量。

要测试你的代码，请实现 `adapters.run_grpo_train_step` 。然后运行 `uv run pytest -k test_grpo_train_step_standard_on_policy` ，确认测试通过。

4.3 实验

现在我们准备好搭建完整的 GRPO 训练循环了，它应当：

- 加载模型和数据集
- 初始化日志记录（例如 Wandb）、vLLM 服务器、优化器
- 执行 RL 训练循环

我们在下面提供了一些推荐超参数。如果你的训练脚本正确，那么对大多数随机种子而言，使用这些超参数训练时，你应该会看到验证奖励随时间上升。

```
n_train_examples = 6400
n_val_examples = 1024
num_rollout_steps = 200
learning_rate = 1e-5
rollout_batch_size = train_batch_size = 256
group_size = 8
gradient_accumulation_steps = 32
sampling_temperature = 1.0
sampling_max_tokens = 512
max_grad_norm = 1.0
optimizer = torch.optim.AdamW(
    policy.parameters(), lr=learning_rate, betas=(0.9, 0.95), weight_decay=0.0
)
```

注意，`rollout_batch_size` 和 `train_batch_size` 统计的是回答数而不是提示数。因此 `rollout_batch_size = train_batch_size = 256` 意味着 32 个提示、每个提示 8 条 rollout。

作为一个合理的默认设置，你可以每 10 个 rollout 批次在验证集上评估一次。由于 CoT/RL 评估噪声可能较大，请确保至少在 1024 个验证样本上评估。除了记录上一小节描述的指标之外，记录 rollout 以便定性地了解模型在做什么也很有用。一个合理的默认做法是每 40 个 rollout 批次

记录一次当前 rollout 批次的训练 rollout。训练样本可以随意使用 `data/gsm8k/train.jsonl` ，验证样本可以使用 `data/gsm8k/test.jsonl` 。

由于策略梯度估计量的方差可能很大，而且 RL 具有自我强化（self-reinforcing）的特性，RL 训练曲线往往有很大的方差。因此，在你确信脚本正确之后，我们会要求你用 4 个随机种子运行你的 RL 循环。即便脚本在给定随机种子时不能完全复现也没关系；你也可以直接把实验运行 4 次。在绘图时，请以某种方式展示各次运行之间的方差（即不要只画均值）。一些合理的做法包括：

- 绘制 4 次运行的均值，同时把各次运行的实际曲线也直接画出来（为了可读性，可以把实际曲线放到单独一张图上，或使用更低的透明度、更细的线宽或虚线）。
- 绘制均值并附带阴影置信区间：具体而言，对每一步计算样本均值和样本标准差，则一个合理的区间为 $\mu \pm 1.96 \cdot \sigma/\sqrt{n}$，其中 n 为随机种子的数量。
- 绘制每一步的均值以及最小值和最大值。

问题（`grpo_experiments_standard_on_policy` ）：用 GRPO 提升 OLMo-2-0425-1B 在 GSM8K 上的性能（2 B200 小时）（10 分）

(a) 编写一个脚本，在给定模型名称、提示词、训练集文件路径、验证集文件路径、采样超参数和训练超参数的情况下运行 GRPO 训练循环。从高层次看，脚本首先要初始化 vLLM 服务器、wandb 日志、数据集、模型和优化器；然后训练循环会重复以下步骤：与 vLLM 服务器同步权重、生成训练 rollout、对训练批次做策略梯度更新，并定期在验证集上检查性能（同时记录生成结果）。

交付要求：一个在 GSM8K 任务和 OLMo-2-0425-1B 模型上运行 GRPO（标准、在策略）的脚本。

(b) 将脚本运行大约 50 步，确认验证奖励在提升，且 rollout 随时间推移看起来是合理的。注意：奖励一开始接近零是正常现象；取决于随机种子，可能需要几步之后我们才会采样到非零奖励的 rollout，模型才开始改进。

交付要求：能让你确信脚本正确的证据（例如：验证奖励逐步提升、rollout 看起来合理）。

(c) 在确信脚本正确之后，使用上面给出的超参数、以 4 个随机种子，在 GSM8K 上运行 OLMo-2-0425-1B。你应当使用零样本（zero-shot）的 `r1_zero` 提示词。如果代码正确，你应该会看到验证奖励随训练提升。

记录以下指标，并绘制它们随时间变化的曲线，同时注明各次运行之间的方差（例如绘制各次运行的均值/标准差或最小值/最大值）：

- 损失。
- 梯度范数。
- token 熵。
- 训练奖励（总奖励、格式奖励）。

- 验证奖励（总奖励、格式奖励）。
- 验证集平均回答长度。
- 其他任何你认为有助于调试的内容。

另请定期记录 rollout 并观察它们：回答是否随着训练而改进？

交付要求：为每个指标写几句话的点评并附一张图，说明它在训练过程中如何变化、各次运行之间方差有多大。

交付要求：训练前后若干个模型 rollout 示例。

交付要求：一个在随机种子间取平均后、最终验证准确率不低于 25% 的流程。

现在我们有了一个能跑通的脚本，可以开始调超参数和做消融实验了。我们先从学习率入手，它通常是对训练影响最大的超参数。

问题（`grpo_learning_rate`）：调整学习率（4 B200 小时）（3 分）

从推荐超参数出发，对学习率做一次扫描（至少包含一个小于推荐默认值的学习率和一个大于推荐默认值的学习率），并汇报最终验证奖励（如果优化器发散，请注明发散）。根据你在上一部分观察到的方差大小，自行决定每次训练运行要使用多少个随机种子。在作业的其余部分，如果你愿意，可以随意使用你调好的学习率来代替推荐默认值。

交付要求：最终验证奖励随学习率变化的曲线图，并附上几句话的点评。

另一个会影响 RL 动力学的选择是提示词：由于 RL 会给模型自己的 rollout 加权，提示词能够塑造模型在训练中探索哪些 rollout。下面我们来看看提示词选择的影响。

问题（`grpo_prompt_ablation`）：提示词消融（4 B200 小时）（3 分）

使用 `question_only` 提示词和 `r1_zero_three_shot` 提示词运行 GRPO 脚本，各用几个随机种子。与上面零样本 `r1_zero` 提示词的结果相比，这些提示词表现如何？哪些提示词的平均奖励最高、方差最低？在其他记录的指标上是否还有其他系统性的差异？基于各次运行之间的方差，你对你的结论有多大把握？

交付要求：点评文字，并为所引用的指标配图。

5 RL 算法变体

GRPO 是语言模型 RL 的流行选择，但它的算法选择在各种论文中存在争议。在本作业中，我们将学习这些选择背后的一些理论论证，并进行受控实验，亲眼看看哪些算法最好（至少对我们的模型与数据集组合而言）。本节讨论在策略（on-policy）设定，下一节讨论离策略（off-policy）算法。

5.1 Dr. GRPO

在推导 GRPO 的过程中，你可能已经注意到一系列选择，它们使得 GRPO 策略梯度估计量不再具有” 正确” 的期望 $\nabla_\theta J_\theta$，其中 J_θ 是模型的期望奖励。这两个选择分别是：标准差优势归一

化，以及序列长度归一化。我们要考虑的第一个消融（在 Dr. GRPO 论文 [Z. Liu et al., 2025] 中有相关论证）就是撤销这些选择：去掉标准差归一化，并且用一个常数来除总损失，而不是先用每条序列的长度对它归一化。记这个常数归一化因子为 Z，则 Dr. GRPO 梯度估计量为

$$\hat{g} \leftarrow \frac{1}{Z}\sum_{i=1}^{B}\sum_{j=1}^{G}\sum_{t=1}^{\operatorname{len}\left(y^{(i,j)}\right)}\left(r\left(y^{(i,j)} \mid x^{(i)}\right)-\mu_i\right)\nabla_\theta \log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right), \tag{33}$$

其中 Z 通常设为训练批量大小乘以最大生成长度（即 $Z = BGL$，其中 L 为最大生成长度（在我们的设置中是 512））。

问题（think_about_length_normalization ）：思考长度归一化（1 分）

在做任何实验之前，请思考下面两种做法的区别：按每条序列的长度对每条序列做归一化，与用同一个常数对所有序列做归一化。两种方法各有什么优缺点？是否存在某些特定设置或例子，使得其中一种方法看起来更好？

交付要求：几句话的讨论。

问题（compute_group_normalized_rewards_drgrpo ）：Dr. GRPO 组归一化（0.5 分）

交付要求：更新你的方法 `compute_group_normalized_rewards` ，使其支持 `advantage_normalizer = "none"` 。虽然 Dr. GRPO 使用 `baseline = "mean"` ，但在本题中，请同时添加对 `baseline = "none"` 的支持。

要测试你的代码，请实现 `adapters.run_compute_group_normalized_rewards` 。然后用 `uv run pytest -k compute_group_normalized_rewards_drgrpo` 运行测试，确保你的实现通过测试。

问题（aggregate_loss_across_microbatch_constant ）：Dr. GRPO 损失聚合（0.5 分）

交付要求：更新你的方法 `aggregate_loss_across_microbatch` ，使其支持 `loss_normalization = "constant"` 。

要测试你的代码，请实现 `adapters.run_aggregate_loss_across_microbatch` 。然后运行 `uv run pytest -k test_aggregate_loss_across_microbatch_constant` ，确保测试通过。

5.2 拒绝微调

我们要研究的下一个消融是一个简单得多的算法，通常称为” 拒绝微调”（rejection fine tuning，RFT）或” 专家迭代”（expert iteration，EI）。顾名思义，该算法的做法是：采样一大批 rollout，保留其中正确的那些，然后在这些正确的 rollout 上做监督微调（即最小化下一个词预测的对数损失）。具体而言，我们的 RFT 梯度为

$$\hat{g} \leftarrow \nabla_\theta\left[\frac{1}{Z}\sum_{i=1}^{B}\sum_{j=1}^{G}\mathbf{1}\{r\left(y^{(i,j)} \mid x^{(i)}\right)=1\}\sum_{t=1}^{\operatorname{len}\left(y^{(i,j)}\right)}\log \pi_\theta\left(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\right)\right], \tag{34}$$

其中 Z 是一个常数归一化因子（与 Dr. GRPO 中一样），$\mathbf{1}\{r(y^{(i,j)} \mid x^{(i)}) = 1\}$ 是一个指示函数，只保留正确的回答。在下面的问题中，我们会思考 RFT 梯度是否真的是策略梯度（即它是否真的

在优化期望奖励 J_θ），以及它与 GRPO 的关系。与之前一样，在把它实现为 PyTorch 损失时，请返回该目标的相反数，使得最小化该损失等价于对这个估计量做梯度上升。

问题（`think_about_rft`）：思考 RFT（2 分）

回忆一下，RFT 目标（采用常数归一化）为

$$J_\theta = \frac{1}{Z}\sum_x \sum_{j=1}^{G} \mathbf{1}\{r(y^{(j)} \mid x) = 1\} \log \pi_\theta\Big(y^{(j)} \mid x\Big), \tag{35}$$

其中 x 是提示，每个 $y^{(j)}$ 表示一条回答 $y^{(j)} \overset{\text{iid}}{\sim} \pi_\theta(\cdot \mid x)$，$G$ 是我们从策略中采样的生成数量，Z 是我们的常数归一化因子，r 是奖励函数。于是 RFT 梯度由 $\nabla_\theta J_\theta$ 给出。

作为对比，在策略（on-policy）的 Dr. GRPO 策略梯度估计量（即采用常数归一化、且不做标准差归一化的 GRPO）为

$$\frac{1}{Z}\sum_x \sum_{j=1}^{G} \Big(r(y^{(j)} \mid x) - \mu\Big) \nabla_\theta \log \pi_\theta\Big(y^{(j)} \mid x\Big), \tag{36}$$

其中 μ 表示组均值 $\mu = \frac{1}{G}\sum_{j=1}^{G} r(y^{(j)} \mid x)$。假设我们的奖励函数 r 是二值的，请比较并对照这两个目标。它们有相同的期望吗？你预期哪一个方差更低？直观上，是否存在某些情形让你更倾向于其中某一个？

交付要求：几句话的讨论。

5.3 MaxRL

最后，我们将研究一种名为最大似然强化学习（Maximum Likelihood Reinforcement Learning）的新方法，也称为 MaxRL [F. Tajwar et al., 2026]。GRPO 选择用组内标准差归一化优势，Dr. GRPO 选择什么都不除，而 MaxRL 则提出除以组均值。这一选择带来如下梯度估计量：

$$\hat{g} \leftarrow \frac{1}{Z}\sum_{i=1}^{B}\sum_{j=1}^{G}\sum_{t=1}^{\text{len}\left(y^{(i,j)}\right)} \frac{r\left(y^{(i,j)} \mid x^{(i)}\right) - \mu_i}{\mu_i} \nabla_\theta \log \pi_\theta\Big(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}\Big). \tag{37}$$

注意，在原始的 MaxRL 论文中，他们不是除以常数归一化因子 Z，而是除以批次中的 token 总数 $\sum_{i=1}^{B}\sum_{j=1}^{G} \text{len}\left(y^{(i,j)}\right)$（这不再是常数，而是依赖于批次）。在本作业中，我们改用常数归一化因子，这样相对于其他基线引入的可变因素更少，更容易分离出" 用 μ_i 代替 std_i 做归一化" 所带来的影响。

在下面的问题中，我们会用一些基础数学来探究不同优势归一化方式的影响。直观地说，除以平均奖励 μ_i 平均而言会给更难的提示赋更大的权重——与简单提示相比，这些提示的 μ_i 往往更低。我们可以通过定义期望奖励的一个重加权版本来把这一直觉形式化：

$$J_{\theta,w} = \mathbb{E}_{x\sim\rho}[w(x, \text{stopgrad}(\pi_\theta))\, \mathbb{E}_{y\sim\pi_\theta}\, r(y \mid x)], \tag{38}$$

其中 $w(x, \text{stopgrad}(\pi_\theta))$ 对提示 x 重新加权，例如基于当前策略在 x 上的期望奖励所刻画的难度 $\eta(x) = \mathbb{E}_{y\sim\pi_\theta}\, r(y \mid x)$，而 `stopgrad` 表示我们不通过该重加权函数计算梯度。在下面的问题中，我们将为目前见过的每种优势归一化方式推导出对应的难度重加权。

问题（derive_difficulty_reweightings ）：推导不同优势归一化方式所诱导的难度重加权（6 分）

回忆一下，在对提示聚合之后，标准策略梯度为

$$\nabla_\theta J_\theta = \nabla_\theta \mathbb{E}_{x\sim\rho}[\mathbb{E}_{y\sim\pi_\theta}\, r(y \mid x)]\,, \tag{39}$$

其中 ρ 是提示 x 上的提示分布，π_θ 表示我们的策略，$r(y \mid x)$ 表示 y 是否是 x 的正确回答。在本题中，我们将推导各个 GRPO 变体所优化的代理目标（surrogate objective），这些代理目标具有如下形式：

$$\nabla_\theta J_{\theta,w} = \nabla_\theta \mathbb{E}_{x\sim\rho}[w(x, \mathrm{stopgrad}(\pi_\theta))\, \mathbb{E}_{y\sim\pi_\theta}\, r(y \mid x)] \tag{40}$$

其中 w 是对提示的某种重加权（我们不通过 w 求导）。

(a) 回忆一下，Dr. GRPO 策略梯度估计量为

$$\mathbb{E}_{x\sim\rho}\left[\frac{1}{Z}\sum_{j=1}^{G}\Big(r(y^{(j)} \mid x) - \mu\Big)\,\nabla_\theta \log \pi_\theta\Big(y^{(j)} \mid x\Big)\right] \tag{41}$$

其中 μ 表示组均值 $\mu = \frac{1}{G}\sum_{j=1}^{G} r(y^{(j)} \mid x)$。取常数归一化因子 $Z = G$ 并令组大小 $G \to \infty$，该估计量等价于优化哪个重加权函数 w 对应的代理目标 $J_{\theta,w}$？

交付要求：一个由问题参数的某个子集表示的表达式，并附上几句论证。

(b) 在采用常数归一化的前提下，GRPO 估计量与 Dr. GRPO 的不同之处在于它还要除以标准差：

$$\mathbb{E}_{x\sim\rho}\left[\frac{1}{Z}\sum_{j=1}^{G}\frac{r(y^{(j)} \mid x) - \mu}{\mathrm{std}}\nabla_\theta \log \pi_\theta\Big(y^{(j)} \mid x\Big)\right], \tag{42}$$

其中 std 表示组标准差。取常数归一化因子 $Z = G$ 并令组大小 $G \to \infty$，该估计量等价于优化哪个重加权函数 w 对应的代理目标 $J_{\theta,w}$？

交付要求：一个由问题参数的某个子集表示的表达式，并附上几句论证。

(c) MaxRL 则改为除以组均值，即

$$\mathbb{E}_{x\sim\rho}\left[\frac{1}{Z}\sum_{j=1}^{G}\frac{r(y^{(j)} \mid x) - \mu}{\mu}\nabla_\theta \log \pi_\theta\Big(y^{(j)} \mid x\Big)\right]. \tag{43}$$

取常数归一化因子 $Z = G$ 并令组大小 $G \to \infty$，该估计量等价于优化哪个重加权函数 w 对应的代理目标 $J_{\theta,w}$？

交付要求：一个由问题参数的某个子集表示的表达式，并附上几句论证。

问题（think_about_advantage_normalization ）：思考优势归一化（2 分）

在做任何实验之前，请思考下面几种做法的区别：用组标准差对组内优势做归一化、用组均值做归一化，或者完全不做优势归一化。各种方法各有什么优缺点？是否存在某些特定设置或例子，使得其中一种方法看起来更好？

交付要求：几句话的讨论。

问题（compute_group_normalized_rewards_maxrl ）：MaxRL 组归一化（0.5 分）

交付要求：更新你的方法 compute_group_normalized_rewards ，使其支持 advantage_normalizer = "mean" 。与 advantage_normalizer = "std" 一样，你应当在归一化分母上加上 advantage_eps ，以避免除以零。

要测试你的代码，请实现 adapters.run_compute_group_normalized_rewards 。然后用 uv run pytest -k compute_group_normalized_rewards_maxrl 运行测试，确保你的实现通过测试。

5.4 实验

现在我们准备好做实验来探究这些不同的策略梯度估计量了。首先，我们需要更新 grpo_train_step 方法以支持这些变体，它们的各个组件我们已在上面实现。具体设置如下：

- GRPO_constant ：baseline = "mean" 、advantage_normalizer = "std" 、loss_normalization = "constant"
- Dr_GRPO ： baseline = "mean" 、 advantage_normalizer = "none" 、 loss_normalization = "constant"
- RFT ： baseline = "none" 、 advantage_normalizer = "none" 、 loss_normalization = "constant"
- MaxRL （采用常数归一化）： baseline = "mean" 、 advantage_normalizer = "mean" 、 loss_normalization = "constant"

为了加速训练，请注意：一旦算出了每条序列的归一化优势，优势为零的序列就无需再传入模型，因为它们对梯度的贡献是零。由于我们的奖励是二值的，序列的优势为零的情况有两种：

- 若 baseline = "mean" ，当一组内所有序列的奖励都相同时，这些序列的优势为零。
- 若 baseline = "none" ，任何奖励为零的序列优势都为零。

在剪枝掉零优势序列之后，我们还可以按相同的比例减少 gradient_accumulation_steps （即保持 microbatch_size 不变）；例如，如果一半的序列优势为零，我们可以做 $k/2$ 次梯度累积步而不是 k 次。这个优化对 RFT 尤其有帮助，因为 RFT 会有相当多奖励为零的序列。但你要在数学和实现上都小心，确保剪枝后的版本计算出的梯度与未剪枝版本相同。

问题（grpo_train_step_variants_on_policy ）：GRPO 训练步变体（2.5 分）

交付要求：更新你的 grpo_train_step 方法，使其支持全部在策略变体（仍然保持 importance_reweighting_method = "none" ）。这些选项包括 baseline: Literal["mean", "none"] 、advantage_normalizer: Literal["std", "none", "mean"] 和 loss_normalization: Literal["sequence", "constant"] 。此外，为了加速训练，你的方法应避免把零优势序列传入模型。

注意，对于 `baseline = "none"` ，不正确的样本得到的奖励为零，因此在损失中的权重也为零，所以没有必要把它们传入模型。你可以利用这一点来调整实现并加速训练。

要测试你的代码，请实现 `adapters.run_grpo_train_step` 。然后运行 `uv run pytest -k test_grpo_train_step_variants_on_policy` ，确认测试通过。

训练步实现完毕之后，我们就可以做实验来观察各方法的表现，并判断它们是否优于标准 GRPO。你应当使用与之前运行相同的超参数，以保证各次运行之间可以相互比较。

关于调超参数的说明：如果我们的目标是判断某个方法是否优于另一个方法，那么在实验上最稳妥的做法是为每个方法分别调超参数。然而，超参数调优非常昂贵，而本作业没有那么多算力。另一种更省钱的选项是：为基线调好超参数，然后把这套超参数用于新方法（例如在我们的情形中，我们在标准 GRPO 上调好了学习率，并在下面的实验中使用这个学习率）。在这种做法下，如果新方法优于基线，我们仍然可以断言它更好，因为次优的学习率给出的是新方法在正确调参后性能的下界；但反过来，如果新方法表现更差，我们不能断言它比基线差，因为我们并没有为它正确地调超参数。

问题（`grpo_experiments_variants_on_policy`）：比较不同 RL 算法的性能（8 B200 小时）（10 分）

在保持超参数与你的标准 GRPO 训练运行一致的前提下（就你选择的学习率而言），运行以下变体，每种变体使用 4 个随机种子。你应当使用零样本（zero-shot）的 `r1_zero` 提示词。

- `GRPO_constant` ：标准 GRPO，但损失聚合采用常数归一化，而不是序列归一化。
- `Dr_GRPO` ：`GRPO_constant` 但 `advantage_normalizer = "none"` 。
- `RFT` ：`GRPO_constant` 但 `advantage_normalizer = "none"` 且 `baseline = "none"` 。注意，对这个方法来说，无需把不正确的样本传入被训练的模型，这应当能加快训练。
- `MaxRL` ：`GRPO_constant` 但 `advantage_normalizer = "mean"` 。注意，原始 MaxRL 实现按每个微批次的 token 数对损失做归一化，而我们改用常数归一化。

与你之前运行的标准 GRPO 相比，这些方法表现如何？哪些方法的平均性能最好，哪些方法的方差最低？你认为哪些方法会从更多的超参数调优中受益？基于各次运行之间的方差，你对自己的结论有多大把握？

交付要求：点评文字，并为所引用的指标配图。

6 离策略 RL

到目前为止，我们的实验和算法都是完全在策略（fully on-policy）的，也就是说，每个梯度估计量使用的都是直接从被更新的模型中采出的样本（即每个推理批次只做 1 次训练步，且训练批量大小等于推理批量大小）。在本节中，我们将研究离策略 RL（off-policy RL）：它在每个推理批次上做多次训练步，有望加速训练，但代价是潜在的不稳定性与更高的算法复杂度。

6.1 重要性重加权

在最朴素的策略梯度版本中，我们采样一批回答，然后在这批回答上做一次大批量的梯度步。但我们可以期待：如果把这一批切成一系列小批量（minibatch），并在每个小批量上做一次训练步，模型也许能训练得更快。这种做法称为" 离策略 RL"，与我们最初" 每个推理批次做一次训练步" 的" 在策略 RL" 形成对比。

这种做法被称为离策略 RL 的原因在于：做完第一个小批量步之后，当前策略就不再等于推理时所用的策略了。此时样本是" 离策略"（off-policy）或" 过期"（stale）的；它们并非来自我们想为之计算策略梯度的当前策略。用记号表示，如果把推理策略记为 π_0、当前策略记为 π_θ，则有

$$\mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_0(y|x)}[r(y\mid x)\nabla_\theta\log\pi_\theta(y\mid x)]\neq\mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_\theta(y|x)}[r(y\mid x)\nabla_\theta\log\pi_\theta(y\mid x)]\,. \tag{44}$$

左边是我们的" 朴素"（naive）离策略估计量：我们假装回答 y 来自当前策略，并机械地套用同样的步骤来计算策略梯度。这个式子说明，这个朴素估计量的期望是不正确的。

纠正这一偏差的一种方法是使用重要性重加权（importance reweighting），它给出如下估计量：

$$\mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_0(y|x)}\left[\frac{\pi_\theta(y\mid x)}{\pi_0(y\mid x)}\,r(y\mid x)\,\nabla_\theta\log\pi_\theta(y\mid x)\right], \tag{45}$$

其中我们应用了重要性权重 $\frac{\pi_\theta(y|x)}{\pi_0(y|x)}$。这些权重的效果是：上调那些对当前策略 π_θ 仍然相关的 y 的权重，同时下调 $\pi_\theta(y\mid x)$ 较低的" 过期" 样本的权重。不难看出，修改后的表达式具有正确的期望：

$$\mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_0(y|x)}\left[\frac{\pi_\theta(y\mid x)}{\pi_0(y\mid x)}\,r(y\mid x)\,\nabla_\theta\log\pi_\theta(y\mid x)\right]=\mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_\theta(y|x)}[r(y\mid x)\nabla_\theta\log\pi_\theta(y\mid x)]\,. \tag{46}$$

缺点是重要性重加权会增大估计量的方差：如果重要性权重 $\frac{\pi_\theta(y|x)}{\pi_0(y|x)}$ 可以大到 C，我们的方差也可能放大 C 倍。这个问题使得朴素的重要性重加权无法直接用于语言模型——语言模型的重要性权重是 $\mathrm{len}(y)$ 个因子的乘积，随回答长度呈指数增长：

$$\frac{\pi_\theta(y\mid x)}{\pi_0(y\mid x)}=\prod_{t=1}^{\mathrm{len}(y)}\frac{\pi_\theta(y_t\mid x,y_{<t})}{\pi_0(y_t\mid x,y_{<t})}. \tag{47}$$

所以，与" 朴素" 的无加权离策略估计量（偏差大，但没有重加权引入的额外方差）相比，这个重加权估计量零偏差但方差大。下面介绍的方法将介于两者之间，提出各种启发式方法来处理这一偏差—方差权衡。

6.2 PPO/GRPO 风格的重要性重加权与裁剪

6.2.1 token 级重加权

标准做法最早由 PPO [J. Schulman et al., 2017] 提出，也被 GRPO [Z. Shao et al., 2024] 采用：做 token 级的重要性重加权，而不是序列级重加权。这一选择带来如下" 无裁剪 token 级重加权" 估计量：

$$\mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_0(y|x)}\left[r(y\mid x)\sum_{t=1}^{\mathrm{len}(y)}\frac{\pi_\theta(y_t\mid x,y_{<t})}{\pi_0(y_t\mid x,y_{<t})}\,\nabla_\theta\log\pi_\theta(y_t\mid x,y_{<t})\right]. \tag{48}$$

注意，这个表达式与我们上面看到的" 有原则的" 序列级估计量并不相同，后者可以改写为（按时间步展开）：

$$\mathbb{E}_{x\sim\rho}\mathbb{E}_{y\sim\pi_0(y|x)}\left[r(y\mid x)\left(\prod_{t=1}^{\mathrm{len}(y)}\frac{\pi_\theta(y_t\mid x,y_{<t})}{\pi_0(y_t\mid x,y_{<t})}\right)\sum_{t=1}^{\mathrm{len}(y)}\nabla_\theta\log\pi_\theta(y_t\mid x,y_{<t})\right]. \tag{49}$$

在深入分析这个修改后的表达式在做什么之前，先注意一点：重加权项不再是关于 $\mathrm{len}(y)$ 指数级的，这会大幅降低方差。

与作业前面的各节一样，为了更好地理解这个估计量，我们要推导它所优化的代理目标 [T. Degris et al., 2013]。具体而言，令 $\tilde{\pi}_t$ 表示这样一个代理策略：除第 t 步从 π_θ 采样之外，所有其他时间步都从旧策略 π_0 采样：

$$\tilde{\pi}_t(y\mid x)=\left(\prod_{s=1}^{t-1}\pi_0(y_s\mid x,y_{<s})\right)\pi_\theta(y_t\mid x,y_{<t})\left(\prod_{s=t+1}^{\mathrm{len}(y)}\pi_0(y_s\mid x,y_{<s})\right). \tag{50}$$

那么，如果我们假设所有回答的长度都相同、均为 L，可以证明 token 级重加权优化的正是每个 $\tilde{\pi}_t$ 下的期望奖励，即

$$J_\theta^{\mathrm{token}}=\mathbb{E}_x\left[\sum_{t=1}^{L}\mathbb{E}_{y\sim\tilde{\pi}_t(y|x)}[r(y\mid x)]\right]. \tag{51}$$

这一事实可以直接通过用重要性重加权项改写该代理目标得到：

$$\begin{aligned}\nabla_\theta J_\theta^{\mathrm{token}}&=\nabla_\theta\mathbb{E}_x\left[\sum_{t=1}^{L}\mathbb{E}_{y\sim\tilde{\pi}_t(y|x)}[r(y\mid x)]\right]\\&=\nabla_\theta\mathbb{E}_x\left[\sum_{t=1}^{L}\mathbb{E}_{y\sim\pi_0(y|x)}\left[\frac{\pi_\theta(y_t\mid x,y_{<t})}{\pi_0(y_t\mid x,y_{<t})}\,r(y\mid x)\right]\right]\\&=\mathbb{E}_x\left[\mathbb{E}_{y\sim\pi_0(y|x)}\left[\sum_{t=1}^{L}\frac{\pi_\theta(y_t\mid x,y_{<t})}{\pi_0(y_t\mid x,y_{<t})}\,r(y\mid x)\,\nabla_\theta\log\pi_\theta(y_t\mid x,y_{<t})\right]\right],\end{aligned} \tag{52}$$

其中最后一步用到了我们在本作业中反复使用的对数导数技巧（log derivative trick）。

这个代理目标直观地展示了 token 级重加权引入的偏差。第一，对第 t 项来说，前缀 $y_{<t}$ 是从 π_0 而不是从 π_θ 采出的，因此它可能无法代表来自当前模型的前缀。第二，当我们从当前策略 π_θ 采样出 token y_t 之后，后缀 $y_{>t}$ 却是从 π_0 而不是从 π_θ 采出的。换句话说，当该目标评估动作 y_t 会如何影响后续步骤时，它依据的是过期策略 π_0 而非当前策略 π_θ 的行为。这两种偏差都会被序列级重加权纠正，因为后者会正确地对前缀和后缀重新加权（代价是方差增大）。直观地说，π_θ 离 π_0 越远，这些偏差就越严重。

问题（`derive_surrogate_objectives`）：推导重要性重加权方法所优化的代理目标（2 分）

在上面我们看到，token 级重要性重加权优化的是这样一组代理策略下的期望奖励：除一个位置外，其余位置都从旧策略采样。假设我们转而考虑一种" 成对"（pairwise）的重要性重加权方法，其策略梯度估计量为

$$\sum_{t=1}^{L/2}\frac{\pi_\theta(y_{2t-1}\mid x,y_{<2t-1})\,\pi_\theta(y_{2t}\mid x,y_{<2t})}{\pi_0(y_{2t-1}\mid x,y_{<2t-1})\,\pi_0(y_{2t}\mid x,y_{<2t})}\,r(y\mid x)\,\nabla_\theta[\log(\pi_\theta(y_{2t-1}\mid x,y_{<2t-1})\,\pi_\theta(y_{2t}\mid x,y_{<2t}))] \tag{53}$$

其中 π_θ 是我们的当前策略，π_0 是我们的过期采样策略，且 $y \sim \pi_0(y \mid x)$。这个估计量在优化什么代理目标？

交付要求：一个由问题参数的某个子集表示的表达式，并附上推导过程。

6.2.2 裁剪

除了 token 级重加权之外，PPO 和 GRPO 引入的另一个启发式方法是重要性权重裁剪（importance weight clipping）。如上所述，π_θ 离 π_0 越远，重要性重加权的方差问题就越严重；而我们的代理目标告诉我们，token 级重加权的偏差也会越发严重。因此，为了让离策略 RL 保持稳定，我们需要保证 π_θ 与 π_0 足够接近。

确保 π_θ 接近 π_0 的最简单办法是减少每个推理批次的训练步数，但有些推理批次本可以支持更多步。为了从算法中榨取性能，我们希望推理批次允许做多少步就做多少步。朝着这个目标，有一类方法会裁剪重要性权重过大或过小的项。

近期的论文在裁剪应当如何实现的细节上意见不一，但在本作业中，我们将实现来自 PPO 的裁剪方法——这一方法延续到了 GRPO，似乎是主流做法。首先，把 token 级重加权与标准 GRPO 目标结合起来，得到如下损失函数：

$$J_\theta^{\text{GRPO-off-policy-noclip}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\operatorname{len}\left(y^{(i,j)}\right)}\sum_{t=1}^{\operatorname{len}\left(y^{(i,j)}\right)}\frac{r\left(y^{(i,j)} \mid x^{(i)}\right)-\mu_i}{\operatorname{std}_i}\,\frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_0(y_t \mid x, y_{<t})}, \tag{54}$$

其中样本取自我们的过期推理策略 π_0。你可以验证，对这个目标求微分得到的正是我们上面推导的 token 重加权策略梯度估计量，只不过用的是 GRPO 优势和序列归一化。

我们准备给这个目标加上裁剪了。令 $A^{(i,j)} = \frac{r\left(y^{(i,j)}|x^{(i)}\right)-\mu_i}{\operatorname{std}_i}$ 表示提示 i 上回答 j 的优势，令 $w_t^{(i,j)} = \frac{\pi_\theta(y_t|x,y_{<t})}{\pi_0(y_t|x,y_{<t})}$ 表示第 t 个重要性重加权项，则该目标的裁剪版本为

$$J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\operatorname{len}\left(y^{(i,j)}\right)}\sum_{t=1}^{\operatorname{len}\left(y^{(i,j)}\right)}\min\Big(A^{(i,j)}w_t^{(i,j)},\ A^{(i,j)}\operatorname{clip}\Big(w_t^{(i,j)},[1-\varepsilon,\,1+\varepsilon]\Big)\Big), \tag{55}$$

其中 $\operatorname{clip}(w,[1-\varepsilon,1+\varepsilon]) = \min(\max(w,1-\varepsilon),1+\varepsilon)$ 把重要性权重裁剪到区间 $[1-\varepsilon,1+\varepsilon]$ 内。这个目标还有一个稍微更直观的写法 [J. Achiam, 2018]：

$$J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\operatorname{len}\left(y^{(i,j)}\right)}\sum_{t=1}^{\operatorname{len}\left(y^{(i,j)}\right)}\begin{cases}\min\left(w_t^{(i,j)},1+\varepsilon\right)A^{(i,j)} & \text{if } A^{(i,j)}\geq 0\\ \max\left(w_t^{(i,j)},1-\varepsilon\right)A^{(i,j)} & \text{if } A^{(i,j)}<0\end{cases}. \tag{56}$$

在这个目标下，模型会被激励去上调高优势动作的权重，直到它给该动作的权重超过旧策略 $1+\varepsilon$ 倍为止；在那一刻，min 会走 $1+\varepsilon$ 那条路，对它求微分得到零梯度。负优势的动作也类似：模型会被激励去下调该动作的权重，直到它给该动作的权重低于旧策略 $1-\varepsilon$ 倍为止，此时该动作的梯度变为零。于是，取梯度后我们得到如下策略梯度估计量：

$$\nabla_\theta J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\operatorname{len}\left(y^{(i,j)}\right)}\sum_{t=1}^{\operatorname{len}\left(y^{(i,j)}\right)}\operatorname{mask}_t^{(i,j)}\ w_t^{(i,j)}\,A^{(i,j)}\,\nabla_\theta\log\pi_\theta(y_t \mid x, y_{<t}) \tag{57}$$

配套一个掩码函数

$$\text{mask}_t^{(i,j)} = \begin{cases} \mathbf{1}\{w_t^{(i,j)} < 1+\varepsilon\} & \text{if } A^{(i,j)} \geq 0 \\ \mathbf{1}\{w_t^{(i,j)} > 1-\varepsilon\} & \text{if } A^{(i,j)} < 0 \end{cases} \tag{58}$$

它对正优势掩掉过大的重要性权重，对负优势掩掉过小的重要性权重。

现在我们有了在损失函数中实现 `importance_reweighting_method = "noclip"` 和 `importance_reweighting_method = "grpo"` 所需的全部数学，它们分别对应 $J_\theta^{\text{GRPO-off-policy-noclip}}$ 和 $J_\theta^{\text{GRPO-off-policy-clip}}$。与在策略情形一样，请在损失实现中返回目标的相反数，使得梯度下降执行预期的梯度上升更新。

问题（`compute_policy_gradient_loss_off_policy`）：带 token 级重加权的离策略策略梯度（1 分）

交付要求：更新你的方法 `compute_policy_gradient_loss`，使其支持 `importance_reweighting_method = "noclip"` 或 `"grpo"`，二者会用到 `old_log_probs` 和 `cliprange` 参数。`old_log_probs` 表示生成 rollout 的模型下每个 token 的对数概率，你需要在训练脚本中添加几行代码来计算它。`cliprange` 表示裁剪强度参数 ε。

要测试你的代码，请实现 `adapters.run_compute_policy_gradient_loss`。然后运行 `uv run pytest -k test_compute_policy_gradient_loss_off_policy`，确保测试通过。

注意：虽然 PPO 中引入裁剪的动机是防止当前策略偏离旧策略太远，但我们也可以把它看作降低重要性重加权估计量方差的手段，因为我们压扁了过大的重要性权重项。如果我们不那么在意偏离旧策略、想在更新上更激进，那么一个简单得多、也直接得多的控方差办法是：直接在梯度估计量中对重要性权重做上界裁剪：

$$\hat{g} \leftarrow \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\frac{1}{\text{len}\big(y^{(i,j)}\big)}\sum_{t=1}^{\text{len}\left(y^{(i,j)}\right)} \min\Big(w_t^{(i,j)}, 1+\varepsilon\Big)\, A^{(i,j)}\, \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}), \tag{59}$$

与 PPO/GRPO 不同的是，对那些被上调超过 $1+\varepsilon$ 倍的动作，我们仍然保留非零梯度。这个更简单的裁剪流程由 CISPO [MiniMax et al., 2025] 提出（注意：他们还按每组的 token 数而不是每条序列的长度做归一化，但为简单起见，我们这里写出的是序列归一化的目标）。你也可以选择性地上手试一下 CISPO，并把它与其他裁剪方法做比较。

6.3 GSPO

如上所述，token 级重加权是有偏的，因为它忽略了前缀与后缀的重加权。在 GSPO 论文 [C. Zheng et al., 2025] 中，作者发现 GRPO 不稳定，并主张应当改做序列级重加权。为了绕开序列级重要性权重是 L 个因子的乘积这一事实，他们干脆把这个表达式取 $\frac{1}{L}$ 次幂。换句话说，他们用整条序列上重要性权重的几何平均（而不是乘积）来做重加权。得到的损失函数如下：

$$J_\theta^{\text{GRPO-off-policy-gspo}} = \frac{1}{BG}\sum_{i=1}^{B}\sum_{j=1}^{G}\min\Big(A^{(i,j)}s^{(i,j)},\ A^{(i,j)}\,\text{clip}\Big(s^{(i,j)}, [1-\varepsilon,\, 1+\varepsilon]\Big)\Big), \tag{60}$$

其中 $s^{(i,j)}$ 是一个在所有时间步之间共享的序列级重要性权重：

$$s^{(i,j)} = \left(\prod_{t=1}^{\operatorname{len}(y^{(i,j)})} \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_0(y_t \mid x, y_{<t})} \right)^{\frac{1}{\operatorname{len}(y^{(i,j)})}}. \tag{61}$$

注意，如果不取 $\frac{1}{\operatorname{len}(y^{(i,j)})}$ 次幂、也不做裁剪，这个目标就退化为序列级重要性重加权的策略梯度损失。取几何平均与裁剪都以偏差为代价降低了方差。另一点值得一提的是：在求梯度时，几何平均的幂次会自然地给出序列长度归一化（为简洁起见，下面的式子忽略了裁剪）：

$$\begin{aligned} \nabla_\theta J_\theta^{\text{GRPO-off-policy-gspo}} &= \frac{1}{BG} \sum_{i=1}^{B} \sum_{j=1}^{G} A^{(i,j)} \nabla_\theta s^{(i,j)} \\ &= \frac{1}{BG} \sum_{i=1}^{B} \sum_{j=1}^{G} A^{(i,j)} s^{(i,j)} \frac{1}{\operatorname{len}(y^{(i,j)})} \sum_{t=1}^{\operatorname{len}(y^{(i,j)})} \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}). \end{aligned} \tag{62}$$

所以，如果你想把 GSPO 应用到你喜欢的某个常数归一化 RL 算法上（选做），你需要修改序列级重要性权重中的指数。

问题（`think_about_importance_reweighting`）：思考重要性重加权（2 分）

考虑离策略 RL 的三种重要性重加权策略：(a) 不做重要性重加权；(b) PPO/GRPO 风格的、带裁剪的 token 级重要性重加权；(c) GSPO 风格的、使用几何平均的、带裁剪的序列级重要性重加权。在偏差与方差的权衡上，各种方法分别位于谱系的哪个位置？你能想到某些情形，使得直觉上某一种方法会比其他方法更好吗？

交付要求：几句话的点评。

在下面的问题中，我们将实现 GSPO 损失。请记住要以数值稳定的方式计算几何平均。

问题（`compute_policy_gradient_loss_off_policy_gspo`）：带序列级重加权的离策略策略梯度（1 分）

交付要求：更新你的方法 `compute_policy_gradient_loss`，使其支持 `importance_reweighting_method = "gspo"`，它会用到 `old_log_probs` 和 `cliprange` 参数。`old_log_probs` 表示生成 rollout 的模型下每个 token 的对数概率，你需要在训练脚本中添加几行代码来计算它。`cliprange` 表示裁剪强度参数 ε。与上面一样，请返回目标的相反数，使得最小化该损失等价于做梯度上升。

要测试你的代码，请实现 `adapters.run_compute_policy_gradient_loss`。然后运行 `uv run pytest -k test_compute_policy_gradient_loss_off_policy_gspo`，确保测试通过。

6.4 实验

现在我们拥有了更新 `grpo_train_step` 方法以支持离策略训练所需的全部组件。

问题（`grpo_train_step_off_policy`）：离策略 GRPO 训练步（2.5 分）

交付要求：更新你的 `grpo_train_step` 方法，使其支持离策略参数（`importance_reweighting_method`、`old_log_probs`、`cliprange`）。现在该函数应当支持全部范围的参数。

要测试你的代码，请实现 adapters.run_grpo_train_step 。然后运行 uv run pytest -k test_grpo_train_step_off_policy ，确认测试通过。

把训练改成离策略真的能更快吗？我们会在多大程度上损失稳定性？哪种重加权/裁剪方法最好？我们将在下面的实验中探究这些问题。请保持超参数与你之前的 RL 运行一致，唯一的例外是这些运行现在应当是 32 倍离策略的（即每个推理批次做 32 次训练步）：

```
rollout_batch_size = 256
train_batch_size = 8
gradient_accumulation_steps = 1
```

此外，作为默认值，你可以使用下面这组裁剪范围取值（即上面推导中的 ε），它们取自原始论文。当然，你也欢迎自行调整这些取值。

- offpolicy_clip : cliprange = 0.2
- offpolicy_gspo : cliprange = 3e-4

注意，你还需要在训练脚本中添加几行代码来计算 old_log_probs ，以便传入你的重加权损失。

问题（grpo_experiments_off_policy ）：比较不同离策略算法的性能（8 B200 小时）（10 分）

在保持超参数与你的标准 GRPO 训练运行一致的前提下（就你选择的学习率而言），运行以下变体，每种变体使用 4 个随机种子。你应当使用零样本的 r1_zero 提示词。如果你愿意，也可以从上一小节中挑选你最喜欢的 RL 算法变体来代替标准版本。

- offpolicy_naive : 不再让推理与训练批量大小相等（均为 256），而是把训练批量大小缩减为推理批量大小的 1/32（$\frac{256}{32} = 8$）。记住要按相同的比例减少梯度累积步数，以便我们继续充分利用 GPU。保持 importance_reweighting_method = "none" 。
- offpolicy_noclip : offpolicy_naive 加上 importance_reweighting_method = "noclip" 。
- offpolicy_clip : offpolicy_naive 加上 importance_reweighting_method = "grpo" 。
- offpolicy_gspo : offpolicy_naive 加上 importance_reweighting_method = "gspo" 。

除了你已经在记录的指标之外，请确保同时记录裁剪比例（clip fraction）。

与你之前运行的完全在策略 GRPO 相比，这些方法表现如何？改为离策略对训练稳定性以及各次运行之间的方差有什么影响？两种裁剪方法在裁剪比例上有何差异，哪一个看起来更稳定？你认为哪些方法会从更多的超参数调优中受益？基于各次运行之间的方差，你对自己的结论有多大把握？

交付要求：点评文字，并为所引用的指标配图。

7 尝试你自己的策略梯度估计量

问题（try_your_own ）：尝试你自己的策略梯度估计量（10 分）

你已经见过文献中提出的各种各样的策略梯度估计量，以及它们背后的一些理论。现在轮到你自己提出一个了！这里有一些建议：

- 你可以尝试一个不同的优势估计量，从而诱导出对提示的不同重加权。
- 你可以尝试一种不同的重要性重加权策略。
- 你可以在优势估计量中尝试一个不同的奖励基线。

请提出你自己的策略梯度估计量。然后在相同的 RL 设定（OLMo-2-0425-1B 在 GSM8K 上）下运行它，并与我们已经尝试过的方法作为基线进行比较，使用多个随机种子并保持问题参数固定，以使结果可以相互比较。请相对已有方法只改动一处（也就是说，本作业覆盖的方法中至少要有一个，使得你的方法相对它只改变了一件事）。

你提出的改动背后的理论依据或直觉是什么？你的改动能否击败这些基线？

交付要求：点评文字，并为所引用的指标配图；如有助于论证你的方法，可选地附上数学推导。

参考文献（Bibliography）

- J. Li *et al.*, "DataComp-LM: In search of the next generation of training sets for language models." [Online]. Available: https://arxiv.org/abs/2406.11794
- T. OLMo *et al.*, "2 OLMo 2 Furious." [Online]. Available: https://arxiv.org/abs/2501.00656
- K. Cobbe *et al.*, "Training Verifiers to Solve Math Word Problems." [Online]. Available: https://arxiv.org/abs/2110.14168
- DeepSeek-AI *et al.*, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning." [Online]. Available: https://arxiv.org/abs/2501.12948
- W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention." 2023.
- Z. Liu *et al.*, "Understanding R1-Zero-Like Training: A Critical Perspective." [Online]. Available: https://arxiv.org/abs/2503.20783
- OpenAI *et al.*, "OpenAI o1 System Card." [Online]. Available: https://arxiv.org/abs/2412.16720
- Z. Shao *et al.*, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models." [Online]. Available: https://arxiv.org/abs/2402.03300

- J. Achiam, "Spinning Up in Deep Reinforcement Learning," 2018.
- N. Lambert, "Reinforcement Learning from Human Feedback." [Online]. Available: https://rlhfbook.com/
- R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992, doi: 10.1007/BF00992696.
- F. Tajwar *et al.*, "Maximum Likelihood Reinforcement Learning." [Online]. Available: https://arxiv.org/abs/2602.02710
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms." [Online]. Available: https://arxiv.org/abs/1707.06347
- T. Degris, M. White, and R. S. Sutton, "Off-Policy Actor-Critic." [Online]. Available: https://arxiv.org/abs/1205.4839
- J. Achiam, "Simplified PPO-Clip Objective." [Online]. Available: https://drive.google.com/file/d/1PDzn9RPvaXjJFZkGeapMHbHGiWWW20Ey/view
- MiniMax *et al.*, "MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention." [Online]. Available: https://arxiv.org/abs/2506.13585
- C. Zheng *et al.*, "Group Sequence Policy Optimization." [Online]. Available: https://arxiv.org/abs/2507.18071